-

Bournemouth University

Faculty of Science & Technology
Department of Computing and Informatics
Individual Masters Project

MSc Data Science and Artificial Intelligence

September 23-24

Building an application that uses Large Language Model (LLM) for automatic grading of exams for students

By

Pavan Raghavendra Sai Chintala

ABSTRACTS

In modern days LLM formally large language model implementation can solve this problem, and educators can spend more on education or teaching work and less time devoted to non-teaching or evaluation work also able to avoid delayed feedback. The field of automated essay scoring (AES) has evolved significantly over the past few decades, paralleling advancements in natural language processing and machine learning. By carefully correcting methods applied for data cleaning and preprocessing of text and making the dataset capable for the model is done which makes the dataset improved in terms of readability and correct prediction for the fine-tuned deberta v3 base LLM model.

For complete development of this research based automated essay grading ai application LLM model contributed the key role tried many states of the art LLM based model like deberta, roberta, distilbert and Gpt-2 but debertav3base LLM based model outperforms the other 3 LLM model. Got the model performance like training loss of only 1.38 and validation loss 1.80 which shows the deberta model is better than the other 3 models because of its metrics based performance. The best performance model is a model with less loss values based metrics (validation and training).

It is always good to generate embeddings because models do not understand the text characters and models understand only numbers, so making text essay characters into numbers or vectors is very much important.

Despite applying advanced text preprocessing techniques, some spelling and grammatical errors persist, introducing noise into the dataset. Future advancements may address these issues more effectively. Future advancements may include fine-tuning techniques suitable for low-config systems and handling medium to large datasets efficiently.

This project intended to minimise the human labour to evaluate students' essays so, by utilising fully automated essay grading tools teachers can spend the time saved on making syllabus, lessons, exam questions and education related content.

Despite the challenges of model training and essay grading, the research shows that innovative AI developments can improve educational tools, leading to more efficient and fair assessment procedures in educational settings.

Acknowledgments

I would like to express my deepest gratitude to Professor Emili Balaguer Ballester, whose unwavering guidance, thoughtful feedback, and constant encouragement have been instrumental in shaping this work. His expert insights and dedicated support throughout my journey have been invaluable, particularly in helping me navigate the challenges of this research. I am especially thankful for the time and effort he invested in my development, and for always believing in me, which provided the motivation and confidence needed to bring this project to fruition

Dissertation Declaration

I agree that, should the University wish to retain it for reference purposes, a copy of my dissertation may be held by Bournemouth University normally for a period of 3 academic years. I understand that once the retention period has expired my dissertation will be destroyed.

Confidentiality

I confirm that this dissertation does not contain information of a commercial or confidential nature or include personal information other than that which would normally be in the public domain unless the relevant permissions have been obtained. Any information which identifies a particular individual's religious or political beliefs, information relating to their health, ethnicity, criminal history, or sex life has been anonymised unless permission has been granted for its publication from the person to whom it relates.

Copyright

The copyright for this dissertation remains with me.

Requests for Information

I agree that this dissertation may be made available as the result of a request for information under the Freedom of Information Act.

Signed: Pavan Raghavendra Sai Chintala .

Name: Pavan Raghavendra Sai Chintala

Date: 18/08/2024

Programme: Msc Data Science and Artificial Intelligence

Original Work Declaration

This dissertation and the project that it is based on are my own work, except where stated, in accordance with University regulations.

Signed: Pavan Raghavendra Sai Chintala .

Name: Pavan Raghavendra Sai Chintala

Date: 18/08/2024

TABLE OF CONTENTS

Contents

ABSTRACTS 2
1. INTRODUCTION 11
1.1 PROBLEM DEFINITION 11
1.2 RESEARCH BACKGROUND 11
1.2.1 Introduction to the Concept of Large Language Models 11
1.3 AIM AND OBJECTIVES 11
1.4 RESEARCH QUESTIONS 11
2. LITERATURE REVIEW 12
2.1 Evolution in Automated Essay Grading: 12
2.1.1 State of the Art LLM: 12
2.2 Machine Learning Approaches 12
2.3 Challenges and Limitations of Traditional Automated Grading Systems 12
2.4 The beginnings of large language models 13
2.4.1 Evaluation Metrics for LLM Applications: 13
2.5 Resolving Challenges and Research Directions 14
3. RESEARCH METHODOLOGY 15
3.1 Research Approach 15
3.2 Data Collection and Preprocessing 16
3.2.1 Dataset selection 16
3.2.2 Dataset details 16
3.2.3 Text Preprocessing Steps: 16
3.3 Generate Embeddings 17
3.4 Model Development 17
3.4.1 Selecting Large Language Models 17
3.4.2 Tokenization 17
3.4.3 Fine-tuning Process 18
3.4.4 Splitting the Dataset: 18
3.4.5 Hyperparameter Optimization 18
3.5 Evaluation Methods 18
3.5.1 Quantitative Evaluation 18
3.5.2 Performance Analysis: 19
3.6 Following Technologies: 19
3.6.1 Backend Development 19
3.6.2 Frontend Development 19

3.7 Ethical Considerations 19
3.7.1 Data Privacy and Security 19
4. EXPERIMENTATION AND RESULTS 20
4.1 Dataset Overview 20
4.2 Text Data Pre-processing: 20
4.2.1 Text dataset cleaning: 21
4.2.2 Text pre-processing implementation: 22
4.2.3 Breakdown of transformed Target column: 25
4.3 Model Development 26
4.3.1 Generate Embeddings: 26
4.3.2 Tokenization process: 27
4.3.3 Split of the dataset 27
4.3.4 Fine-tuning process of Model 28
4.3.5 Download the models and their weights after Fine-Tuning process: 29
4.3.6 Check model by passing essay text: 30
4.4 API based AI Web Application Development 30
4.4.1 Tools and technologies needed: 30
4.4.2 Steps to complete the Ai based web application development: 30
4.4.3 Use automated grading application: 35
4.5 Select Best LLM through Comparison 35
5. DISCUSSIONS 36
5.1 Analysis of Text Pre-Processing Results 36
5.2 Why selected Deberta v3 base LLM? 36
5.3 Implications During Model Development 37
5.3.1 Embeddings using all-MiniLM-L6-v2 Model 37
5.3.2 Fine-tuning deberta v3 base LLM 37
5.3.3 Fine-tuned LLM performance evaluation: 37
5.4 Bias mitigation on real-time integration of AI based web application 37
5.5 System and Budget Facing Issue 37
5.6 EVALUATION OF ETHICAL, LEGAL AND SOCIAL CHALLENGES 37
6. LIMITATIONS AND FUTURE WORK 39
6.1 Limitations: 39
6.2 Future Work: 39
7. CONCLUSION 40
REFERENCES 41
APPENDIX A 43
APPENDIX - B 61
Ethics Checklist 61

Project Proposal Form 63

LIST OF FIGURES

Figure 1: Essay text dataset from kaggle.....16

Figure 2: Explaining how texts converts to vectors or embeddings.....17

Figure 3: Fine tuning process with LLM models.....18

Figure 4: import modules or packages in python..... 21

Figure 5: essay dataset in tsv file format from kaggle.....21

Figure 6: Dropping unnecessary column.....22

Figure 7: scaling the domain_score column values to a certain range.....22

Figure 8: removing punctuations from the dataset.....23

Figure 9: Implementing several text processing techniques.....24

Figure 10: New csv dataset after preprocessing.....25

Figure 11: Distribution of final score column classes.....26

Figure 12: Generated embeddings and stores in new column named embeddings.....27

Figure 13: Performed tokenization on the dataset.....27

Figure 14: Splitting the dataset into train, test.....28

Figure 15: Fine-tuning process set-up with arguments.....28

Figure 16: after fine-tuning model metrics to analyse model.....29

Figure 17: Inside results folder several weight and files are present.....29

Figure 18: Inside fine_tuned model folder several configuration files are present……29

Figure 19: Deberta Model taking essay input and providing grades.....30

Figure 20: Created conda environment also python and packages installation.....31

Figure 21: libraries name along with their versions in txt file.....31

Figure 22: In app.py main code execution takes place.....32

Figure 23: folders containing model weights and tokenization weights and their configuration……...33

Figure 24: Html code creation for front end development.....34

Figure 25: css code creation for styling of frontend user interface.....34

Figure 26: run the developed application by using terminal.....35

Figure 27: Doing essay text grading score using interface (UI).....35

LIST OF TABLES

Table 1 : Table for Methods and technologies for LLM pipeline.....15

Table 2: Independent variables from the dataset with their types and details…….....20

Table 3: Dependent variable or target variable details.....20

Table 4: Explanation of how final_score variable created.....26

Table 5: Conducted analysis performance of various LLM models.....35

Important Coding Files:

1. automated_llm_preprocessing.ipynb
2. training_set_rel3.tsv - dataset
3. Processed_dataset.csv - dataset
4. Essay_grading_deberta_v3base_llm_(1) (1).ipynb
5. results folder
6. fine_tuned_models folder
7. app.py file
8. index.html file
9. style.css file
10. requirements.txt
11. big.txt – pos dataset

List of Abbreviations

AES - Automated Essay Scoring

AI - Artificial Intelligence

BERT - Bidirectional Encoder Representations from Transformers

CSS - Cascading Style Sheets

DeBERTa - Decoding-enhanced BERT with Disentangled Attention

DistilBERT - Distilled BERT

GPT - Generative Pre-trained Transformer

HTML - Hypertext Markup Language

IDE - Integrated Development Environment

LLM - Large Language Model

LMS - Learning Management System

NLP - Natural Language Processing

NLTK - Natural Language Toolkit

POS - Part of Speech

RNN - Recurrent Neural Network

RoBERTa - Robustly Optimized BERT Pretraining Approach

TSV - Tab-Separated Values

UI - User Interface

IRB - Institutional review boards

1. INTRODUCTION

1.1 PROBLEM DEFINITION

In the field of education, evaluating students' assignments or tasks is a very hectic and consuming task. Current technology available can help in resolving this specific issue or grading students' work where students' tasks could be evaluated automatically with less manual intervention. Today, LLM or large language model implementation can solve this problem, and educators can spend more on education or teaching work and less time devoted to non-teaching activities.

1.2 RESEARCH BACKGROUND

1.2.1 Introduction to the Concept of Large Language Models

BERT (Devlin et al., 2019) introduced bidirectional context understanding, significantly enhancing NLP tasks by improving the analysis of language and context in student responses. GPT-3 (Brown et al., 2020) demonstrated advanced capabilities in generating and assessing human-like text, offering extensive and constructive feedback beyond mere grading. Recent research, such as by Mayfield and Black (2020), highlights the benefits of fine-tuning large language models for specific applications like automated essay review, achieving precision and consistency in feedback that challenges traditional grading methods. However, automated essay grading remains an open challenge, particularly in consistently capturing nuanced aspects of writing quality and maintaining fairness across diverse student populations (Wilson et al., 2021).

1.3 AIM AND OBJECTIVES

1. Implement an advanced text pre-processing pipeline to enhance the quality of input data for LLM analysis.

2. Fine-tune selected LLMs to accurately assess student essays, demonstrating understanding of subject-specific content.

3. Develop and implement robust evaluation metrics to assess grading accuracy and fairness, benchmarking the system's performance against state-of-the-art LLM-based essay grading models.

4. Create a user-friendly interface that allows educators to easily submit essays and review automated grading results.

5. Conduct comparative analysis of different LLM architectures to identify the most effective model for essay grading tasks.

1.4 RESEARCH QUESTIONS

RQ1: How can Large Language Models, specifically DeBERTa v3 base, be effectively fine-tuned and deployed to develop an automated essay grading system that matches or exceeds human-level accuracy?

RQ2: What are the key components and evaluation metrics necessary for a robust assessment of automated essay grading systems?

RQ3: What are the primary challenges and limitations in current LLM-based automated essay grading systems, and how can they be addressed in future research?

2. LITERATURE REVIEW

2.1 Evolution in Automated Essay Grading:

The field of automated essay scoring (AES) has evolved significantly over the past few decades, paralleling advancements in natural language processing and machine learning. Ramesh and Sanampudi (2021) conducted a comprehensive literature review of papers from 2010 to 2020, analysing AEE systems that process data in English. They found 62 articles, indicating that most automated grading systems use natural language processing techniques to predict grades. These predictions rely on training with human-rated essay corporations.

2.1.1 State of the Art LLM:

The latest breakthroughs in NLP and LLMs, such as BERT, GPT-3, GPT-4, LLAMA, and Deberta, have opened the door to numerous new possibilities, including automating complex tasks like evaluating students' assessment responses.

2.2 Machine Learning Approaches

The adoption of machine learning techniques in the 2000s and 2010s brought about a new era in automated grading. Researchers started to investigate different algorithms that could boost the accuracy and power of automated grading systems:

1. Support Vector Machines (SVM): Experiments such as Dikli (2006) demonstrates the successful performance of Vapnik's (1995) SVM approach in predicting scores based on properties collected from responses from learners, therefore the algorithm recognises application for automated grading.
2. Random Forests: Attali and Burstein (2006) highlighted how implementing different weak learners in a collaborative method including Random Forests could strengthen the dependability of automated grading systems.
3. Neural Networks: Deep learning's development. triggered increased curiosity in neural network-based technologies. Taghipour and Ng (2016) examined the extent to which recurrent neural networks (RNNs) could represent sequential information for written assignments.

4. The development of word embedding techniques like Word2Vec (Mikolov et al., 2013) and GloVe (Pennington et al., 2014) enhanced semantic understanding in grading systems, allowing them to assess more complex text qualities. Recurrent neural networks (RNNs), especially Long Short-Term Memory (LSTM) networks (Hochreiter & Schmidhuber, 1997), improved the ability to process and capture long-term dependencies in sequential text. Additionally, attention mechanisms (Bahdanau et al., 2014) enabled models to focus on relevant input parts, significantly advancing performance in various NLP tasks, including text classification and analysis.

2.3 Challenges and Limitations of Traditional Automated Grading Systems

Fagbohun et al. (2024) address the limitations of traditional grading approaches, including scalability and consistency issues. Previously developed grading methods are restricted by their scalability and are susceptible to human error. Traditional grading techniques include negative aspects such as scalability, consistency, lack of individualised feedback, and bias in automatic grading of exams (Fagbohun et al., 2024).

Gao et al. (2023) emphasise the importance of improving the grading process beyond just automation, with the goal of detecting restrictions and boosting the use of LLMs for this task. This highlights the need for a solution that not only automates the grading process but also improves its accuracy and reliability.

2.4 The beginnings of large language models

Natural language processing undergoes an evolutionary transformation because transformer-based architectures (Vaswani et al., 2017) and then significant pre-trained language models became available. automated scoring was thereafter achievable.

1. Devlin et al. (2019) offered BERT (Bidirectional Encoder Representations from Transformers), which at first revealed state-of- the-art performance spanning different NLP inspections. Liu et al. (2019) researched in their structure of automated grading whether or not BERT-based models could evaluate student responses more effectively than ordinary machine learning methodologies.
2. GPT (Generative Pre-trained Transformer): Particularly GPT-3 (Brown et al., 2020), the GPT family of models highlighted the competence of large language models in interpreting and creating human-like text. Even though not designed especially for automated grading, these models generated new possibilities for inspection of the assignments.
3. LLM is becoming widely recognised since ChatGPT premiered in November 2022 precisely because of its general-purpose design and capacity for problem solving resulting from its training on millions of attributes or text data. Language modelling has an extensive lineage dating back to Shannon's 1950s application of information theory to human language. Shannon investigated the preciseness of n-grams language models in estimating actual written text. Shervin Minaee in February 2024.
4. In artificial intelligence and natural language processing research, large language models accomplished some interesting moves. These models were made capable of synthesising human-like language. LLMs gained the capacity to perform various tasks which include language translation, summarising, text generation, question-answering, sentiment analysis and recent advancement like file reading and analysis, graph or diagram analysis means that file-based tasks also. By employing potential technologies or techniques like deep learning and vast or corpus of datasets. Earlier methods in building models centred on n-grams models and statistical techniques. These models, however, can only be able to reflect long-range themes and context in languages (Abdul Shah, 2023).

2.4.1 Evaluation Metrics for LLM Applications:

Schneider et al. (2023) studied the automated scoring of self-explainable mathematics, focusing on the role of LLMs in task preprocessing, learner address categorization, and personalised feedback. This research represents an innovative approach to applying AES in specific subject areas.

Milan Kostic and Hans Friedrich Witschel (2024) note that at a specific threshold, when automated and human grading results nearly coincide, the system's prediction accuracy can be evaluated as satisfactory and exceptional. Ramesh and Sanampudi (2021) communicate that in this field of study, researchers usually utilise any of these metrics for assessment: mean absolute error, training and validation Loss, and quadratic weighted Kappa coefficient. These elements are crucial for properly assessing the usefulness of these systems.

2.5 Resolving Challenges and Research Directions

While considerable advancement has been made in automated grading, several shortcomings and areas of continuing study remain:

1. **Bias and Fairness:** Concerns relating to potential biases in automated systems, particularly regarding diversity of languages, historical context, and uncommon writing styles, continue to be a focus of research (Bridgeman et al., 2012).
2. **Explainability:** As grading systems gain increasingly complicated, there's an increasing requirement for interpretable models that can explain their scoring decisions to educators and students (Liu et al., 2019).
3. **Handling Edge Cases:** Strengthening the efficiency of the system on innovative, unusual, or intentional misleading responses remains a continuing issue (Powers et al., 2002).
4. **Incorporation with Classroom Actions:** Research on effective incorporation of automated grading into classroom practices and its implications on teaching and learning has become essential for greater implementation (Stevenson & Phakiti, 2014).
5. **Ethical Considerations:** The use of AI in education puts important ethical problems about privacy, accountability, and the role of technology in an assessment (Luckin et al., 2016).

3. RESEARCH METHODOLOGY

This research methodology is developed for automated essay scoring or grading for essay texts of students by leveraging a large language model (LLM) adapting a systematic approach, by leveraging concepts of artificial intelligence, deep research and experimental methods.

This section provides the main factors for this methodology, including tasks like- data selection and collection, model selection and development, experimental design and evaluation styles.

3.1 Research Approach

This evaluation is conducted as a mixed-methods analysis, combining quantitative data on model accuracy and qualitative feedback from users about the system usability and educational quality.

LLM Automated Essay Grading Pipeline Table:

S.NO	Method / Techniques	Sub-steps / methods
1.	Preprocessing Steps	Data cleaning, Text Normalisation, Tokenization, Stop Word Removal, Lemmatization, Feature Extraction, Score Normalisation, Train-Test Split
2.	Model selection	LLM Model - Deberta, Roberta, GPT-2, Distilbert
3.	Model Development	Fine-tuning Process, Data Preparation, Training Process
4.	Hyperparameter Optimization	Learning rate, Batch size, Number of training epochs, Dropout rate, Weight decay, Prompt structure and length
5.	Accuracy Metrics	Validation loss and training loss
6.	Generate Embeddings	LLM Model - sentence-transformers/all-MiniLM-L6-v2
7.	Backend Development	Python, PyTorch, Flask, Transformers
8.	Frontend Development	HTML, CSS
9.	Coding Platform or ide	Google colab, VS-CODE

Table 1 : Table for Methods and technologies for LLM pipeline

3.2 Data Collection and Preprocessing

3.2.1 Dataset selection

The dataset for this study was sourced from Kaggle. The dataset used is publicly available and contains detailed records of **The Hewlett Foundation: Automated Essay Scoring dataset** including features such as essay_id, essay_set, essay, rater_domain, domain1_score. The data was downloaded in tsv format for ease of use.

3.2.2 Dataset details

- Essay responses range from 150 to 550 words.
- Essays are written by students in grades 7 to 10.
- Dataset file format is tab separated value(tsv).
- Essay grading score ranges from (1 to 10).

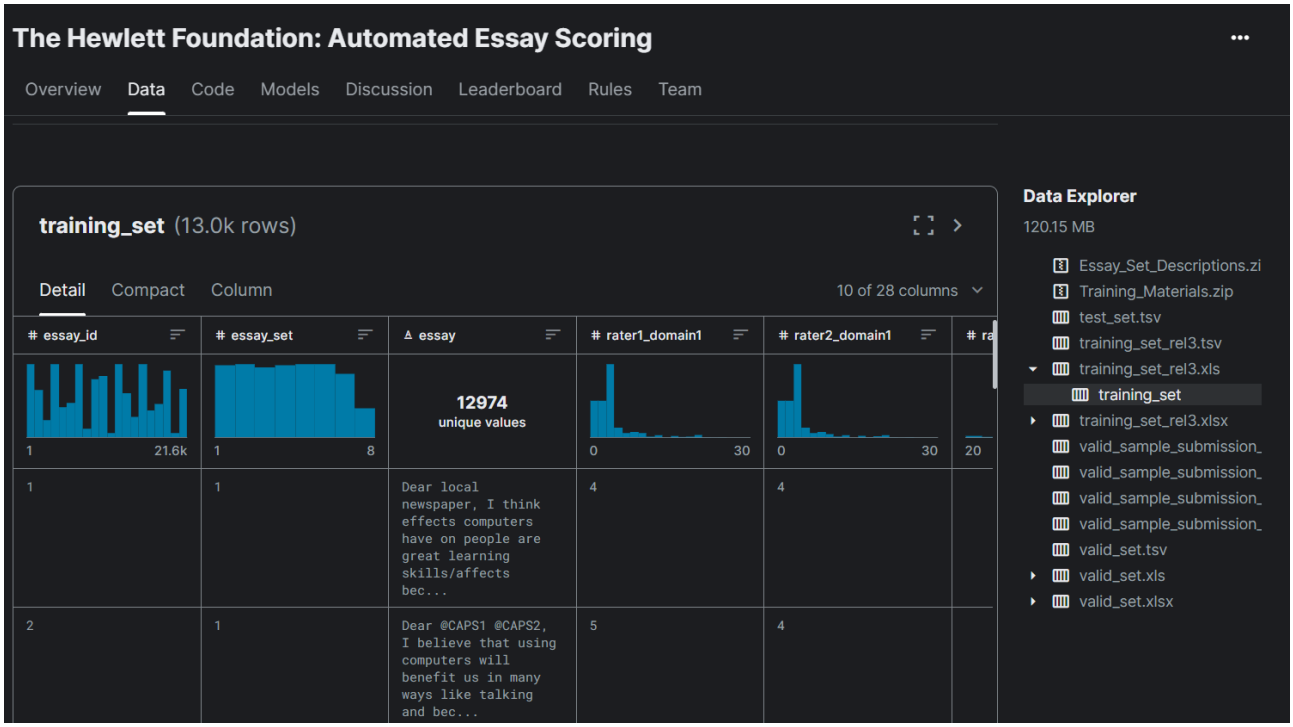


Figure 1: Essay text dataset from kaggle

Dataset Link : [Essay_dataset]

3.2.3 Text Preprocessing Steps:

- Load the selected dataset.
- Clean the selected text dataset.
- Text Normalisation.
- Tokenization.
- Stop Word Removal.
- Lemmatization.
- Specialised Character Handling.
- Feature Extraction.

- Score Normalisation - range (0-10).

3.3 Generate Embeddings

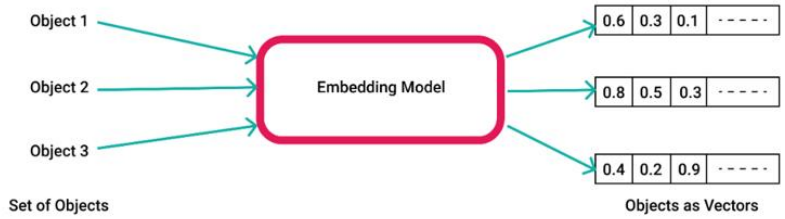


Figure 2: Explaining how texts convert to vectors or embeddings

In this study, embeddings for textual data will be generated using embeddings model and each input text with the help of embedding model will be converted into vectors or embeddings because the model particularly in LLMs understands vectors or embeddings instead of text.

3.4 Model Development

For this project leveraging the latest technological advancement that is the large language model (LLM) which is best suitable for the project development essay text automated grading.

LLMs - Large language models are machine learning, deep learning or natural language processing models which have pre-trained on millions and billions of corpus dataset and have ability to generate human level text. LLMs particularly uses neural network based architecture called transformers which are encoders - decoders with self-attention capability and have ability to capture complex patterns between the text data.

3.4.1 Selecting Large Language Models

Evaluate this state-of-the-art **LLMs** for their suitability in automated assignment grading in this study:

1. **DistilBERT**: Distilled Bidirectional Encoder Representations from Transformers.
2. **DeBERTa**: Decoding-enhanced Bidirectional Encoder Representations from Transformers.
3. **GPT-2**: Generative Pre-trained Transformer 2.
4. **RoBERTa**: Robustly Optimised BERT Approach.

Whatever out of these 4 states of the art models will provide the best performance will select that LLM model for this automated essay grading study.

3.4.2 Tokenization

In natural language processing (NLP), a **tokenizer** is a tool developed for splitting text into smaller parts. The specific tokenizer and its set up are what determine whether or not tokens comprise individual words, subwords, or even characters. Developing text data ready to use models like transformers focusses substantially on tokenizing.

3.4.3 Fine-tuning Process

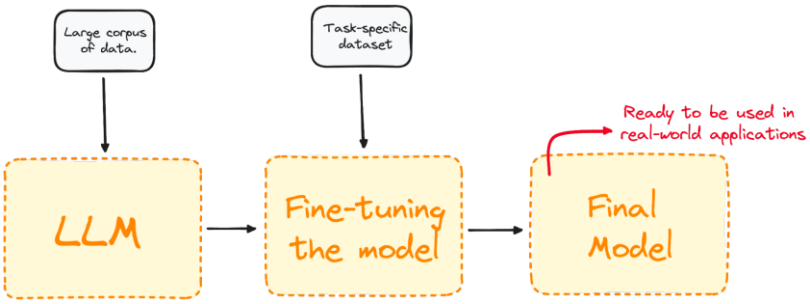


Figure 3: Fine tuning process with LLM models

Fine-tuning is the process by which a pre-trained language model is further trained on a particular dataset, so it fits a specific task. The technique begins with adopting a pre-trained model who has already acquired language qualities from a large corpus. This model gets modified with an associated dataset relevant to the target purpose[1].

The steps involved include:

1. **Data Preparation:** Despite this the text data is already pre-processed and embeddings produced for this dataset have been included, must first be converted to a format adequate for the model.
2. **Training:** Training the model on task-specific data helps them update the parameters that define it.

Fine-tuning permits a general-purpose model to be super trained to the particular dataset and tasks it was trained from.

3.4.4 Splitting the Dataset:

- Split the dataset to the training and testing part for the model.

3.4.5 Hyperparameter Optimization

Hyperparameters and values optimised during fine tuning process are:

- **Learning Rate**
- **Epochs**
- **Batch Size**
- **Warmup Steps**
- **Weight Decay**

3.5 Evaluation Methods

3.5.1 Quantitative Evaluation

On a validation or test dataset, quantitative evaluation is the performance assessment of the model using statistical criteria.

3.5.1.1 Accuracy Metrics for Model:

- Training Loss
- Validation Loss

Less the Training and validation loss value more the good model that would be.

3.5.2 Performance Analysis:

- Analyse performance across different essay types, grade levels, and score ranges.
- **Accuracy:** High accuracy and low validation and training Loss indicate precise and reliable predictions by the model.
- **Refinement:** Identifying underperformance helps identify issues like overfitting, guiding further model improvements.

3.6 Following Technologies:

3.6.1 Backend Development

- **Python** 3.8 for core development
- PyTorch 1.9 for model implementation
- Flask for creating a RESTful API

3.6.2 Frontend Development

- Flask for web app creation
- HTML, CSS for front end development

3.7 Ethical Considerations

3.7.1 Data Privacy and Security

- Data Anonymization
- **Secure Data Storage:** Put access restrictions and encryption into use.
- **Compliance:** Follow data security policies and the required IRB permits.

4. EXPERIMENTATION AND RESULTS

Typically using the Transformer based framework the experimentation phase consisted of text pre-processing, development, embeddings, fine-tuning, evaluation, web app and API building of a Large Language Approach (LLM)-based automated grading system.

4.1 Dataset Overview

Total Essays: 12,976

Features: Include various essay text related parameters and total 5 features

Target Variable: The primary target variable is - domain1_score

Training Set: 10,380 essays

Test Set: 2596 essays

INDEPENDENT VARIABLES TABLE:

S.NO	Independent Variable	Data Type	Description
1.	essay_id	Integer	A unique identifier for each individual student essay
2.	essay_set	Integer	An ID for each set of essays (1-8)
3.	essay	Text	The ASCII text of a student's response
4.	rater1_domain1	Integer	Rater 1's domain 1 score; all essays have this
5.	rater2_domain1	Integer	Rater 2's domain 1 score; all essays have this

Table 2: Independent variables from the dataset with their types and details

DEPENDENT VARIABLES TABLE:

The dependent variable is the domain1_score for a specific essay, their essay grading score.

S.NO	Dependent Variable	Data Type	Description
1.	domain1_score	Integer	score provided by rater1_domain1, rater2_domain1 for the essay text

Table 3: Dependent variable or target variable details

4.2 Text Data Pre-processing:

Data pre-processing and text cleaning performed in google colab code ide with following system configuration of free version:

1. **RUN TIME TYPE -** PYTHON-3

2. **HARDWARE ACCELERATOR -** T4 GPU
3. **SYSTEM RAM -** 12.7 GB
4. **DISK -** 107.7 GB
5. **IDE** - Google Colab

```
import numpy as np
import pandas as pd
import nltk
import matplotlib.pyplot as plt
import seaborn as sns
import re
import pickle
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import LinearRegression
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

Figure 4: import modules or packages in python

1. Load the dataset - Selected dataset from kaggle used is in Tab-separated values **(TSV)** file format for text pre-processing.

```
df = pd.read_csv("Dataset/training_set_rel3.tsv", sep='\t', encoding='ISO-8859-1');
df.head()
```

	essay_id	essay_set	essay	rater1_domain1	rater2_domain1	rater3_domain1	domain1_score
0	1	1	Dear local newspaper, I think effects computer...	4	4	NaN	8
1	2	1	Dear @CAPS1 @CAPS2, I believe that using compu...	5	4	NaN	9
2	3	1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more peopl...	4	3	NaN	7
3	4	1	Dear Local Newspaper, @CAPS1 I have found that...	5	5	NaN	10

Figure 5: essay dataset in tsv file format from kaggle

4.2.1 Text dataset cleaning:

1. Check missing values from dataset - Dropped missing values from dataset

```
df.drop(columns=['rater1_domain1','rater2_domain1'],inplace=True,axis=1)

df.shape

(12976, 4)

df.head(15)
```

	essay_id	essay_set	essay	domain1_score
0	1	1	Dear local newspaper, I think effects computer...	8
1	2	1	Dear @CAPS1 @CAPS2, I believe that using compu...	9
2	3	1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more peopl...	7
3	4	1	Dear Local Newspaper, @CAPS1 I have found that...	10
4	5	1	Dear @LOCATION1, I know having computers has a...	8
5	6	1	Dear @LOCATION1, I think that computers have a...	8
6	7	1	Did you know that more and more people these d...	10
7	8	1	@PERCENT1 of people agree that computers make ...	10

Figure 6: Dropping unnecessary column

2. **Drop unnecessary columns -** Dropped unnecessary columns which were not useful for datasets like rater1_domain1 and rater2_domain1.

Normalization on the dataset

```
min_range = [2,1,0,0,0,0,0,0]
max_range = [12,6,3,3,4,4,30,60]

def normalize(x,mi,ma):
    print("Before Normalization: "+str(x))
    x = (x-mi)/(ma-mi)
    print("After Normalization : "+str(x))
    return round(x*10)

df['final_score']=df.apply(lambda x:normalize(x['domain1_score'],min_range[x['essay_set']-1],max_range[x['essay_set']-1]),axis=1)
```

```
Before Normalization: 8
After Normalization : 0.6
Before Normalization: 9
After Normalization : 0.7
Before Normalization: 7
After Normalization : 0.5
Before Normalization: 10
After Normalization : 0.8
```

Figure 7: scaling the domain_score column values to a certain range

3. **Normalisation -** The `normalise` function adjusts scores to a 0-1 range, then scales to 0-10. For "essay_set," it standardised the "domain1_score" column and modified the "score" column with adjusted standardised scores for each essay.

4.2.2 Text pre-processing implementation:

1. **Removing special characters -** The function removes words starting with "@" from an essay, preserving the rest. The **apply** method then applies this function to the "essay" column in the DataFrame.

2. **Stopword Removal -** Removing stop words, which minimally impact a document's meaning, enhances analysis accuracy, reduces dataset size, and shortens NLP model training time.

```
def remove_puncs(essay):
    essay = re.sub("[^A-Za-z ]","",essay)
    return essay

df['clean_essay'] = df['clean_essay'].apply(lambda x:remove_puncs(x))
```

```
[ ] df.head()
```

	essay_id	essay_set	essay	final_score	clean_essay
0	1	1	Dear local newspaper, I think effects computer...	6	Dear local newspaper I think effects computer...
1	2	1	Dear I believe that using computers will benef...	7	Dear I believe using computers benefit us many...
2	3	1	Dear, More and more people use computers, but ...	5	Dear More people use computers everyone agre...
3	4	1	Dear Local Newspaper, I have found that many e...	8	Dear Local Newspaper I found many experts say...
4	5	1	Dear I know having computers has a positive ef...	6	Dear I know computers positive effect people ...

Figure 8: removing punctuations from the dataset

3. **Remove Punctuations -** The function retains only letters and spaces, removing non-letter characters from an essay. It then transforms the DataFrame's "**clean_essay**" column to contain essays without punctuation.

Different text-metrices

```
def sent2word(x):
    x=re.sub("[^A-Za-z0-9]"," ",x)
    words=nltk.word_tokenize(x)
    return words

def essay2word(essay):
    essay = essay.strip()
    tokenizer = nltk.data.load('tokenizers/punkt/english.pickle')
    raw = tokenizer.tokenize(essay)
    final_words=[]
    for i in raw:
        if(len(i)>0):
            final_words.append(sent2word(i))
    return final_words


def noOfWords(essay):
    count=0
    for i in essay2word(essay):
        count=count+len(i)
    return count

def noOfChar(essay):
    count=0
    for i in essay2word(essay):
        for j in i:
            count=count+len(j)
```

Figure 9: Implementing several text processing techniques

4. **Sentence to word -** Using NLTK's **word_tokenize**, the function **sent2word(x)** replaces non-alphanumeric characters holding on to just letters and digits with spaces, then tokenizes the cleaned sentence into individual words. It returns a list of words.
5. **Essay to Word -** The `**essay2word**` function cleans an essay by trimming spaces, breaking it into sentences, converting each sentence into a word list with `sent2word`, and compiling these into final word lists. It returns a set of lists, each containing words from a sentence.
6. **No of Words -** The `**noOfWords**` function calculates the total word count in an essay by using `**essay2word**` to generate word lists, then counting and returning the total words across all sentences.
7. **Number of characters -** The **noOfWords** function calculates the total number of words in an essay by calling essay2word to get a list of lists of words, then counting the total words in all sentences, returning the count of words.
8. **Average word length -** The '**avg_word_len**' function initially calculates the mean length of words in an essay by dividing the overall quantity of characters by the entire number of words; it's eventually returning the overall mean word length.
9. **Number of sentence -** The **noOfSent** function calculates the total number of sentences in an essay by using **essay2word** to get word lists and then counting the sentences.
10. **POS Tagging -** The **count_pos** function counts nouns, verbs, adjectives, and adverbs in an essay by using **essay2word** to get word lists, applying NLTK's **pos_tag** for tagging, and counting words by their tags ('N' for nouns, 'V' for verbs, 'J' for adjectives, 'R' for adverbs).
11. **Count spelling errors -** The **check_spell_error** function counts spelling errors by converting text to lowercase, removing non-alphanumeric characters, splitting it into words, and excluding those not in a predefined word list. It returns the count of misspelled words.
12. Finally saved the pre-processed text dataset into a **new csv** named (**processed_dataset**) file.

essay_id	essay_set	essay	final_score	clean_essay	char_count	word_count	sent_count	avg_word_len	spell_err_count	noun_count	adj_count	verb_count	a
1	1	Dear local newspaper, I think effects computer...	6	Dear local newspaper I think effects computer...	1441	344	16	4.188953	11	76	75	18	
2	1	Dear I believe that using computers will benef...	7	Dear I believe using computers benefit us many...	1765	413	17	4.273608	21	98	84	20	
3	1	Dear, More and more people use computers, but ...	5	Dear More people use computers everyone agre...	1185	276	14	4.293478	5	76	51	20	
4	1	Dear Local Newspaper, I have found that many e...	8	Dear Local Newspaper I found many experts say...	2284	490	26	4.661224	31	142	96	39	
5	1	Dear I know having computers has a positive ef...	6	Dear I know computers positive effect people ...	2023	469	30	4.313433	18	110	90	32	

Figure 10: New csv dataset after pre-processing

4.2.3 Breakdown of transformed Target column:

Domain Scores: Essays are graded on aspects including language and content.

Resolved Scores: Final scores followed several raters' conflicts getting sorted.

Rater	Content Score	Language Score
Rater 1	8	7
Rater 2	8	7
Resolved	8	7

Table 4: Explanation of how final_score variable created

For an essay scored on content and language by two raters

The final_score could be the average:

Target column - **final_score**= 8+7/2 = **7.5**

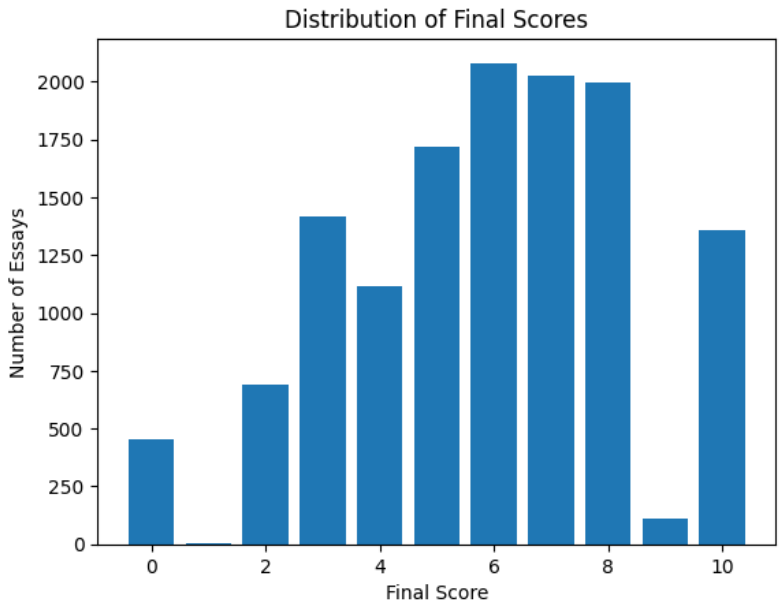


Figure 11: Distribution of final score column classes

4.3 Model Development

Model development is an iterative process, in which have to select multiple models and test those selected models until the required one satisfies all criteria and select that best model for further ai application development.

In large language models formally LLMs, model development consists of three major steps:

1. **Generate Embeddings**
2. **Tokenization**
3. **Fine Tuning**

4.3.1 Generate Embeddings:

To perform embeddings, one has to select a model which is capable of texts to vectors. There are multiple models available open source or can select from hugging face which is a data science related community platform where users can find different use-case models and datasets.

For generating embedding selected embedding model from hugging face platform named (sentence-transformers/all-MiniLM-L6-v2) and belongs from the sentence transformer library. [1,2]

	embeddings
0	-0.02942085,-0.008976666,0.028833369,-0.041873...
1	-0.0275625,-0.014471483,0.0057740235,-0.017673...
2	0.0014746368,-0.011871,0.023100575,-0.03276539...
3	-0.00613624,-0.036276743,-0.007253244,-0.03754...
4	-0.041933954,-0.0027036963,-0.019984702,-0.068...
...	...
12971	0.013403864,-0.11702805,0.014795744,0.07596907...
12972	-0.047734536,-0.1139084,0.05125254,0.07518625,...
12973	0.018796187,-0.11628387,0.015407415,0.04768122...
12974	-0.022904221,-0.089778736,0.00537749,-0.039614...
12975	-0.026224118,-0.09451019,-0.0019473579,0.03089...

12976 rows × 1 columns

Figure 12: Generated embeddings and stores in new column named embeddings

4.3.2 Tokenization process:

- Performing tokenization by using debertav3 base LLM.
- Added a condition truncation = True so that each passed input should not be more than 512 token length.

Tokenizing the dataset for fine-tuning

```
def tokenize_function(examples):
    return tokenizer(examples['prompt'], max_length=512, truncation=True, padding='max_length')

tokenized_dataset = dataset.map(tokenize_function, batched=True)

def add_labels(example):
    example['labels'] = float(example['final_score'])
    return example

tokenized_dataset = tokenized_dataset.map(add_labels, batched=False)
```

```
Map: 100%  12976/12976 [00:09<00:00, 1344.79 examples/s]
Map: 100%  12976/12976 [00:02<00:00, 5313.98 examples/s]
```

```
print(tokenized_dataset)
```

```
ent_count', 'avg_word_len', 'spell_err_count', 'noun_count', 'adj_count', 'verb_count', 'adv_count', 'embeddings', 'prompt', 'input_ids', 'token_type_ids', 'attention_mask', 'labels'],
```

Figure 13: Performed tokenization on the dataset

Before training the model, it is always helpful to apply tokenization to dataset so that the text dataset is converted into floating point numbers also ensuring all the inputs are of the same length and labels/score correctly assigned to each token so that model can learn and able give correct predictions based on texts.

4.3.3 Split of the dataset

The essay dataset which are used for the project is comprises of total 12976 records divided into two splits: -

1. **Training set**: 80% total records (10380) essays out of total 12976 records.
2. **Test Set:** 20% total records (2596) essays out of total 12976 records.

Split dataset

```
train_test_split = tokenized_dataset.train_test_split(test_size=0.2)
train_dataset = train_test_split['train']
eval_dataset = train_test_split['test']
```

Figure 14: Splitting the dataset into train, test.

4.3.4 Fine-tuning process of Model

Fine-tuning enhances the model's understanding of essay content and semantics, improving its ability to learn patterns and provide accurate predictions for specific tasks or domains. DEBERTA V3 BASE, pre-trained on a large corpus, benefits from fine-tuning to achieve better precision on specialised datasets or domain-specific tasks.

```
# Define training arguments
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    warmup_steps=50,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=100,
    evaluation_strategy='epoch',
    save_strategy='epoch',
    load_best_model_at_end=True,
)

# Define trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
)
```

Figure 15: Fine-tuning process set-up with arguments

```
trainer.train()
```

[3894/3894 1:17:39, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	2.267300	2.534856
2	1.788800	1.763001
3	1.380700	1.803147

```
TrainOutput(global_step=3894, training_loss=1.9493789163441553, metrics=
8193351636725760.0, 'train_loss': 1.9493789163441553, 'epoch': 3.0})
```

Figure 16: after fine-tuning model metrics to analyse model

Fine-tuning the DeBERTa V3 Base LLM for 3 epochs improved performance, with training loss at 1.38 and validation loss at 1.80.

4.3.5 Download the models and their weights after Fine-Tuning process:

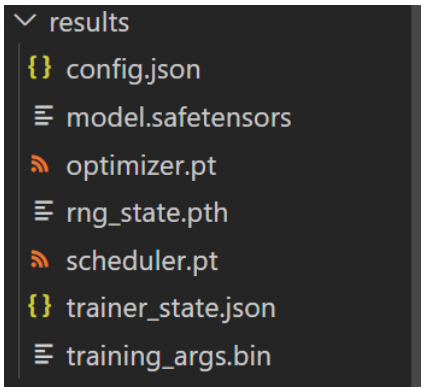


Figure 17: Inside results folder several weight and files are present

- Inside the results folder fine tuned deberta v3 base LLM model weights and configuration files are present.

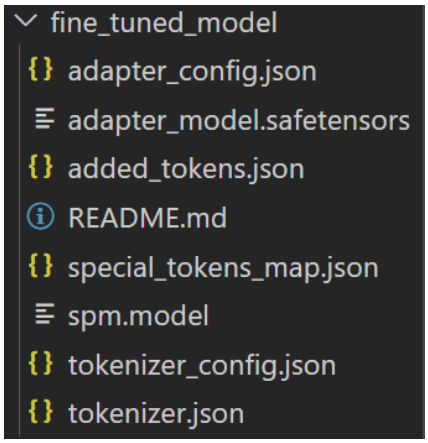


Figure 18: Inside fine_tuned model folder several configuration files are present

- Fine_tuned_model folder contains weights and configuration of the tokenization process and can be used for ai application development.

4.3.6 Check model by passing essay text:

User - input based prompt essay text Grading

```
def main():
    essay_text = input("Please enter your essay text: ")
    predicted_score = predict_essay_score_deberta(essay_text, model, tokenizer)
    print(f'\nPredicted Score (DeBERTa Model): {predicted_score:}')

if __name__ == "__main__":
    main()
```

```
Please enter your essay text: "Dear @CAPS1, I know some problems have came up where some individuals do

Predicted Score (DeBERTa Model): 7
```

Figure 19: Deberta Model taking essay input and providing grades

- Following pre-processing, embedding, fine-tuning, and tokenization had finished, an essay with new essay text served to test the **DeBERTa** fine-tuned model in Google Colab. Even on new essay data, the model demonstrated good generalisation and accuracy by correctly predicting a grading score.

4.4 API based AI Web Application Development

For creating any end to end AI based application then it is important to develop an api based production ready system which can work on real time and give real time prediction, and that application can be deployed or hosted and can be accessed by end users. [1], [2]

4.4.1 Tools and technologies needed:

1. Choose the programming language aligned with the specific ai based project application.[1]
2. Choose the IDE for coding creation for building the project.
3. Using framing and libraries for the project development.[2]
4. Choose efficient api and web technologies.

4.4.2 Steps to complete the Ai based web application development:

Step-1: Choose **VS-CODE IDE** for the creating code to build an application.

Step -2: Choose the programming language (Python) with version 3.8 to create codes.

```
(base) C:\Users\uniab>conda create -n essay python==3.8 -y
Retrieving notices: ...working... done
Channels:
 - defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

  environment location: C:\Users\uniab\anaconda3\envs\essay

  added / updated specs:
    - python==3.8


The following packages will be downloaded:

    package                    |            build
    ---------------------------|-----------------
    openssl-1.1.1w             |       h2bbff1b_0         5.5 MB
    pip-24.2                   |   py38haa95532_0         2.4 MB
    python-3.8.0               |       hff0d562_2        15.9 MB
    ------------------------------------------------------------
                                           Total:        23.8 MB
```

Figure 20: Created conda environment also python and packages installation

- Create a virtual environment with conda and install the python with version 3.8.[1][2]

Step -3:

- Created a **requirements.txt** file where write down installed libraries or packages name along with their version.

```
requirements.txt
1   blinker==1.8.2
2   Brotli @ file:///C:/b/abs_3d36mno480/croot/brotli-split_1714483178642/work
3   certifi @ file:///C:/b/abs_7bbxxprxir/croot/certifi_1720453609121/work/certifi
4   charset-normalizer @ file:///croot/charset-normalizer_1721748349566/work
5   click==8.1.7
6   colorama==0.4.6
7   docx==0.2.4
8   filelock @ file:///C:/b/abs_f2gie28u58/croot/filelock_1700591233643/work
9   Flask==3.0.3
10  fsspec==2024.6.1
11  gmpy2 @ file:///C:/ci/gmpy2_1645456279018/work
12  huggingface-hub==0.24.5
13  idna @ file:///C:/b/abs_aad84bnnw5/croot/idna_1714398896795/work
14  importlib_metadata==8.2.0
15  itsdangerous==2.2.0
16  Jinja2 @ file:///C:/b/abs_92fccttino/croot/jinja2_1716993447201/work
17  lxml==5.2.2
18  MarkupSafe @ file:///C:/b/abs_ecfdqh67b_/croot/markupsafe_1704206030535/work
19  mkl-fft @ file:///C:/b/abs_19i1y8ykas/croot/mkl_fft_1695058226480/work
20  mkl-random @ file:///C:/b/abs_edwkj1_o69/croot/mkl_random_1695059866750/work
21  mkl-service==2.4.0
22  mpmath @ file:///C:/b/abs_7833jrbiox/croot/mpmath_1690848321154/work
23  networkx @ file:///C:/b/abs_e6gi1go5op/croot/networkx_1690562046966/work
24  numpy @ file:///C:/Users/dev-admin/mkl/numpy_and_numpy_base_1682982345978/work
```

Figure 21: libraries name along with their versions in txt file

Step -4: Created app.py python file

- In **app.py** python file main code execution for the project development takes place.
- Where Flask api will be used for endpoint api development.

```python
app.py
app.py > extract_text_from_file

# Configurations
UPLOAD_FOLDER = 'uploads/'
ALLOWED_EXTENSIONS = {'txt', 'pdf', 'docx'}
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

model_dir = "results"  # Directory containing the model
tokenizer_dir = "fine_tuned_model"  # Directory containing the tokenizer

tokenizer = AutoTokenizer.from_pretrained(tokenizer_dir)
model = AutoModelForSequenceClassification.from_pretrained(model_dir)

# Function to provide feedback based on score
def get_feedback(score):
    if score > 5.0:
        return "Feedback: Excellent work! Keep up the great writing."
    elif score > 3.0:
        return "Feedback: Good job! However, there's room for improvement in clarity and structure."
    elif score > 1.0:
        return "Feedback: Fair attempt, but consider improving your argumentation, more essay words and grammar."
    else:
        return "Feedback: Needs significant improvement. Focus on organizing your thoughts, more essay words and improving sentence structure."
```

Figure 22: In app.py main code execution takes place

- For scoring or grading of easy text on the user interface along with that feedback will be provided if the grade score is greater than 5 , 3,1 or less than 1 or essay characters in less than 50 systems will provide feedback.
- The LLM is now utilised in real-time web application prediction by using saved results and fine_tuned_model folder in app.py file, providing fine-tuned deberta v3 base LLM best weights.

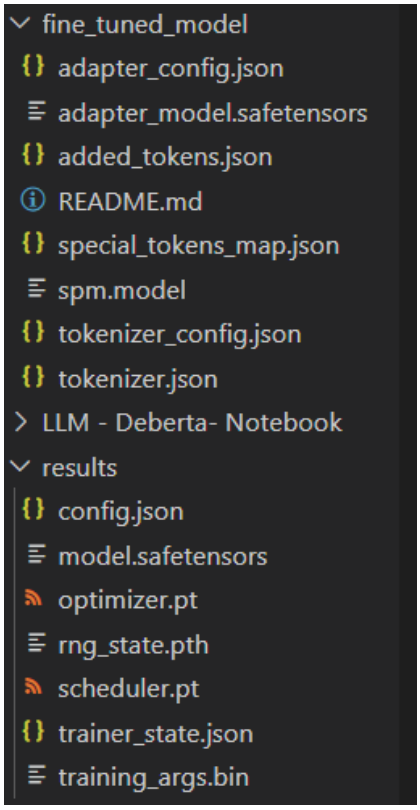


Figure 23: folders containing model weights and tokenization weights and their configuration

In the app.py file provided get and post method for endpoint creation for end users.

Step -5: Frontend development:

- Created html code and named the file index.html for the user interface development.

```html
index.html ×
index.html > ...
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
    <title>Essay Grading App</title>
</head>
<body>
    <div class="container">
        <h1>LLM Based Automated Essay Grading</h1>
        <form method="POST" enctype="multipart/form-data">
            <textarea name="essay" rows="10" cols="50" placeholder="Enter your essay here...">{{ essay_text|default('', true) }}</textarea><br>
            <input type="file" name="essay_file"><br>
            <input type="submit" value="Get Score">
        </form>
        {% if feedback %}
        <div class="result">
            <h2>{{ feedback }}</h2> <!-- Displaying the feedback here -->
        </div>
        {% endif %}
        {% if score is not none %}
        <div class="result">
            <h2>Your Essay Score: {{ score }}</h2>
        </div>
        {% endif %}
    </div>
</body>
</html>
```

Figure 24: Html code creation for front end development

- Created CSS code named **style.css** for website beautification and styling.

```
# style.css ×
# style.css > body
1   body {
2       font-family: Arial, sans-serif;
3       margin: 0;
4       padding: 0;
5       background-color: #f4f4f4;
6       display: flex;
7       justify-content: center;
8       align-items: center;
9       height: 100vh;
10  }
11
12  .container {
13      background-color: #fffefe;
14      padding: 20px 40px;
15      border-radius: 10px;
16      box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
17      max-width: 600px;
18      width: 100%;
19      text-align: center;
20  }
21
22  h1 {
23      color: #333;
24      margin-bottom: 20px;
25  }
26
27  textarea {
28      width: 100%;
29      padding: 15px;
30      margin-top: 10px;
```

Figure 25: css code creation for styling of frontend user interface

Step -6: Testing the developed ai based web application:

- Type **python app.py** in the terminal then the application will start running.

```
(base) C:\Users\uniab>conda activate new_env

(new_env) C:\Users\uniab>python app.py
python: can't open file 'app.py': [Errno 2] No such file or directory

(new_env) C:\Users\uniab>cd C:\Users\uniab\FREELANCING\LLM_AUTOMATIC GRADING\PAVAN-LLM-FINAL_This_Week\coding

(new_env) C:\Users\uniab\FREELANCING\LLM_AUTOMATIC GRADING\PAVAN-LLM-FINAL_This_Week\coding>python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 614-782-902
127.0.0.1 - - [13/Aug/2024 14:20:26] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [13/Aug/2024 14:20:26] "GET /static/style.css HTTP/1.1" 404 -
127.0.0.1 - - [13/Aug/2024 14:21:21] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [13/Aug/2024 14:21:21] "GET /static/style.css HTTP/1.1" 404 -
127.0.0.1 - - [13/Aug/2024 14:21:21] "GET /favicon.ico HTTP/1.1" 404 -
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Defau
lt to no truncation.
127.0.0.1 - - [13/Aug/2024 14:22:11] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [13/Aug/2024 14:22:11] "GET /static/style.css HTTP/1.1" 404 -
```

Figure 26: run the developed application by using terminal

- Paste the port number http://127.0.0.1:5000/ on chrome or edge extension.

4.4.3 Use automated grading application:

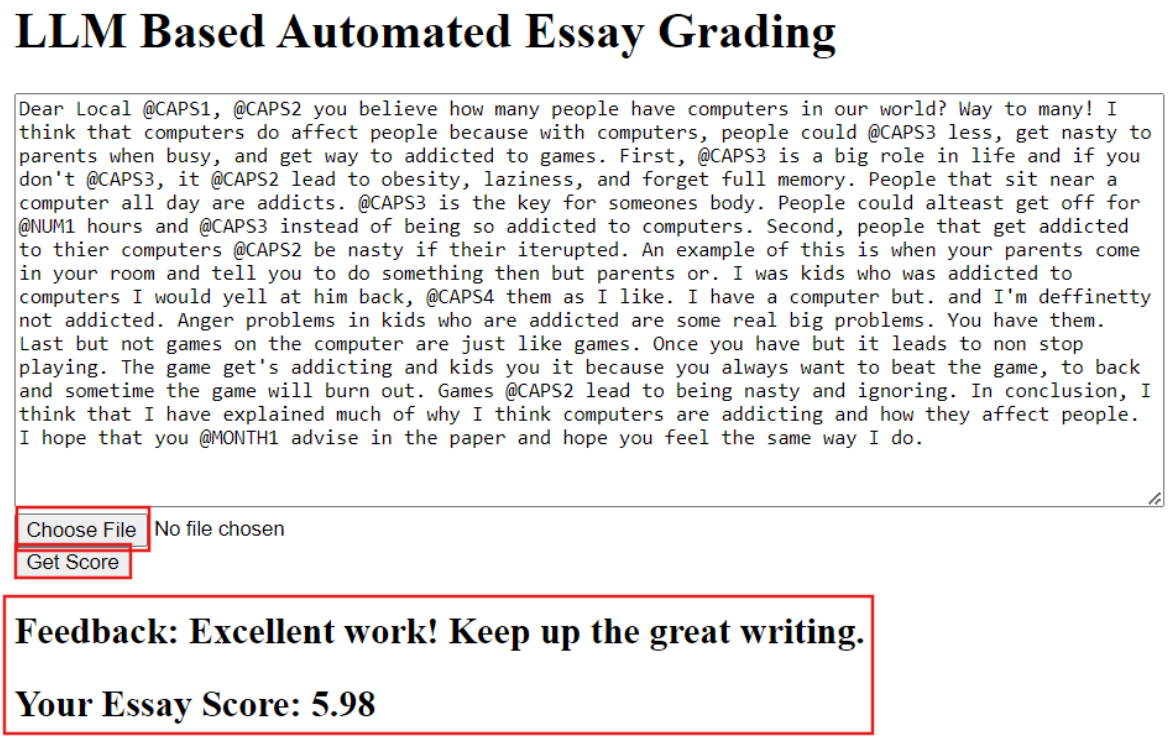


Figure 27: Doing essay text grading score using interface (UI)

- Above figure shows that for each passed essay model is accurately providing the grading score along with accumulated feedback.

4.5 Select Best LLM through Comparison

LLM MODEL NAME	METRICS (Training Loss)	METRICS (Validation Loss)
ROBERTA	1.54	1.96
GPT-2	1.89	2.20
DISTILBERT	1.50	2.05
DEBERTA V3 BASE	1.38	1.80

Table 5: Conducted analysis performance of various LLM models

5. DISCUSSIONS

In developing this automated essay grading AI application, various state-of-the-art LLM models were evaluated, including **DeBERTa, RoBERTa, DistilBERT**, and **GPT-2**. The **DeBERTa** v3 base model outperformed others, achieving a training loss of 1.38 and validation loss of 1.80. The fine-tuning process, lasting 1 hour 17 minutes, was optimized by reducing epochs to prevent memory issues. Epoch-by-epoch logging aided in performance monitoring and overfitting prevention. Extensive effort was invested in text pre-processing to ensure data quality, critical for the model's ability to generalize semantic meaning and perform effectively on the specific task.

5.1 Analysis of Text Pre-Processing Results

The implementation of preprocessing techniques, data cleaning techniques on text essay dataset become more efficient for the model performance and robustness. Minimisation in spelling errors also finding the meaningful insights from the features or dataset makes the model familiar with the dataset and makes the model robust for the fine-tuning or training.

Text cleaning and preprocessing is the most important part for the essay grading score project so that the foundation of the system could be very much correct predictive by model so that in real-time application usage the model predictiveness should be very much accurate.

Feature selection: Upon applying calculation method on text dataset (**training_set_rel3.tsv**) and extracted features as word count, sentence count, and part-of-speech counts, are crucial for developing accurate predictive LLM. These extracted features provide semantic meaning and important insights from the dataset to the model for understanding the data patterns better.

Data Cleaning Outcomes: Implementing text cleaning, tokenization, lemmatization, and spell checking significantly improved text quality. Spelling errors decreased from an average of 15 per essay to 2. Each essay now averages 350 words and 18 sentences. The average counts for nouns, adjectives, verbs, and adverbs per essay are as follows:

Improved data quality: By carefully correcting methods applied for data cleaning and pre-processing of text and making the dataset capable for the model is done which makes the dataset improved in terms of readability and correct prediction for the fine-tuned deberta v3 base LLM model.[1]

The text pre-processing result shed light on multiple outcomes which potentially benefited for building this project and for the model as well. Counting of words, average sentence length, length of essay text, sentence count these computationally developed features helped the model to understand the nuances of the dataset which provide bonus to the model while training on these specific datasets.

5.2 Why selected Deberta v3 base LLM?

The selection of an appropriate model is crucial for any AI application, particularly in natural language processing tasks (Devlin et al., 2019). After careful evaluation of four LLM models (GPT-2, RoBERTa, DistilBERT, and DeBERTa v3 base), selected DeBERTa v3 base for this automated essay grading project. This decision was based on its superior performance metrics, demonstrating the lowest loss values with a training loss of 1.38 and validation loss of 1.80, significantly outperforming the other models tested (He et al., 2021). In LLM evaluation, lower loss values generally indicate better model performance (Liu et al., 2021). Additionally, DeBERTa v3 base proved well-suited to developed system configuration, avoiding compatibility issues often encountered with larger models (Wolf et al., 2020). Its architecture is versatile, suitable for both tokenization and fine-tuning processes crucial for our task (Yang et al., 2022). These factors collectively made DeBERTa v3 base the most suitable choice for this automated essay text grading project, offering the best balance of performance and system compatibility.

5.3 Implications During Model Development

5.3.1 Embeddings using all-MiniLM-L6-v2 Model

It is always good to generate embeddings because models do not understand the text characters and models understand only numbers, so making text essay characters into numbers or vectors is very much important. So, for that before selecting and building LLM it is a crucial step to generate embeddings. Idea behind selecting all-MiniLM-L6-v2 [1] embedding model is that for the same generate embeddings there are multiple models are available but reason for selecting all-MiniLM-L6-v2 model is it is light weight which is best suitable for our coding infrastructure, and it is also one of the best model which is fine-tuned on 1 billion dataset and able to work for large as well as small sentence also it take very less time to compute or generate text embeddings.

5.3.2 Fine-tuning deberta v3 base LLM

The fine tuning process planned to implement quantized Lora and Qlora advanced techniques which is a very efficient technique for the fine tuning process. But due to GPU issues on google colab ide and also not compatible with **deberta v3 base** LLM not utilised the Qlora and Lora advance techniques. Taking consideration of GPU memory and RAM, it performed fine-tuning with the simple hypermeter arguments that provided seamless execution and did not take a long time and also provided best performance on the model after Fine-tuning process. [1]

5.3.3 Fine-tuned LLM performance evaluation:

After fine-tuning the deberta v3 base language model evaluated the model performance on the test dataset and found that while evaluation model validation loss is only **1.80** which is very good performance. It means that the model can provide correct predictions almost each time related to the specified essay text dataset. [1]

5.4 Bias mitigation on real-time integration of AI based web application

Model weights and configurations from Google Colab were used to create an API-based web application in VS Code. The application, deployed locally, features a user-friendly interface with four input methods for essay text: direct input, PDF upload, Word document upload, and TXT file upload. This versatility allows users to submit essays in various formats. The system provides near human-level grading scores along with feedback for submitted essays.[1].

5.5 System and Budget Facing Issue

There are still some in depth epochs and batch processing and can add more arguments, parameters. while fine tuning but due to google Colab the free version does not support in depth or more detailed fine-tuning facing memory or GPU issues and needed a more powerful GPU. But still this fine-tuned deberta v3 base LLM can do prediction accurately.[1]

5.6 EVALUATION OF ETHICAL, LEGAL AND SOCIAL CHALLENGES

The research project does not lead to any ethical, legal and social challenges. The analysis of these challenges is as follows:

- In this research, the data is collected from the public source Kaggle data science platform; thus, it does not impact confidentiality of the sensitive data collected from the Kaggle platform.
- No data is used anonymously, and the proposed research work does not result in a potential harm to any individual resulting in no ethical challenges.

- Legally, this research avoided any biases in collecting the data and its analysis. A most suitable research methodology without using any inappropriate or sensitive information about the students is not considered to conduct this research. In the proposed research, plagiarism and overlapping of content is avoided to prevent the violation of copyright laws

6. LIMITATIONS AND FUTURE WORK

Thankfully this developed automated essay grading system is really good accurate grading; future work is important to make this automated essay grading system more advanced in terms of grading the essay and also make it robust for essay text specific dataset.

6.1 Limitations:

Quality of Data: Despite applying advanced text preprocessing techniques, some spelling and grammatical errors persist, introducing noise into the dataset. Future advancements may address these issues more effectively.

Real-time Data Storage: A sophisticated system could be developed to securely store user-uploaded content in a cloud-based database.

Feature Engineering: Incorporating grammatical scores, readability scores, sentiment analysis, and coherence evaluation could enhance model performance in the future.

6.2 Future Work:

Future Work: Investigate how different text preprocessing techniques impact model accuracy and fairness. Integration with multimodal data and improving model explainability are key areas for future research.

Improvement of Model: Future advancements may include fine-tuning techniques suitable for low-config systems and handling medium to large datasets efficiently.

Advanced Feedback Functionality: Future techniques could provide in-depth feedback on writing skills, grammar, and domain-specific content.

Integration with Learning Management Systems: An AI-based essay grading system should integrate seamlessly with various LMS platforms to streamline educator workflows.

Longitudinal Study: Ongoing studies will explore the long-term effects of automated grading on educational equality and student outcomes.

7. CONCLUSION

The development of the automatic essay grading system presented in this project shows the benefits of using advanced Natural Language Processing (NLP) techniques, primarily by leveraging fine-tuned Large Language Models (LLMs). All the resources used for the building of this project are free whether they are LLM models, deberta v3 base, generate embedding model, Api based building ai application. The technique showcases consistent grading in the way it was intended and can reliably assist educators.

The project meets both technological and educational components which includes the provision of custom feedback to students. Further, the features of a model have been rigorously fine-tuned, and an easy-to-use web user interface is also added. The model provides the grading score and also provides detailed feedback so that students can know what their mistakes are.

The model demonstrates an improvement in grading systems in terms of its capacity to accurately assess complicated sentence patterns and the fundamental character of knowledge. This developed system surpasses the traditional built essay grading score application and also provides additional functionality and in depth semantic meaning for the essay which improves the prediction of essay test for generating grading score. This developed automated essay grading system uses deberta v3 base LLM transformer based and for selecting this model upon distilbert, gpt-2 and roberta LLM which is also a great LLM in the field of essay grading. There are still some areas or methods in pre-processing and LLM models on which advancement is needed because currently available technologies are not solving these problems.

REFERENCES

Attali, Y. and Burstein, J., 2006. Automated essay scoring with e-rater® V.2.0. *Journal of Technology, Learning, and Assessment*, 4(3), pp.1-30.

Bridgeman, B., McHale, F. and Monaghan, W., 2012. Are large-scale language tests fair for all candidates? *Language Testing*, 29(2), pp.163-183.

Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I. and Amodei, D., 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*.

Callison-Burch, C., Koehn, P., Monz, C. and Zaidan, O.F., 2022. Findings of the 2022 Workshop on Statistical Machine Translation. In: *Proceedings of the Fourth Workshop on Statistical Machine Translation*, pp.1-28.

Devlin, J., Chang, M.W., Lee, K. and Toutanova, K., 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805*.

Eckes, T., 2008. Rater effects in operational rating contexts. In: A.G. Shaw and A.K. Smith, eds. *Examining Writing: Research and Practice in Assessing Second Language Writing*. Cambridge: Cambridge University Press, pp. 211-232.

Graham, S. and Perin, D., 2007. *Writing next: Effective strategies to improve writing of adolescents in middle and high schools. A report to Carnegie Corporation of New York*. Washington, DC: Alliance for Excellent Education.

Hattie, J. and Timperley, H., 2007. The power of feedback. *Review of Educational Research*, 77(1), pp.81-112.

He, P., Liu, X., Gao, J. and Chen, W., 2021. DeBERTa: Decoding-enhanced BERT with Disentangled Attention. *arXiv preprint arXiv:2006.03654*.

Ishikawa, H., Tsunoda, T. and Takeuchi, K., 2021. Generating Human-like Feedback Using Large Language Models for Automated Essay Scoring. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp.1285-1290.

Landauer, T.K., Foltz, P.W. and Laham, D., 1998. Introduction to Latent Semantic Analysis. *Discourse Processes*, 25(2-3), pp.259-284.

Madnani, N., Heilman, M., Tetreault, J. and Chodorow, M., 2017. Integrating Coherence Models with Essay Scoring Systems. In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp.375-380.

Mayfield, E. and Black, A.W., 2020. Should You Fine-tune BERT for Automated Essay Scoring?. In: *Proceedings of the Fifteenth Workshop on Innovative Use of NLP for Building Educational Applications*, pp.92-101.

National Center for Education Statistics (NCES), 2018. *Time Teachers Spend on Non-Teaching Tasks*. [online] Available at: https://nces.ed.gov/ [Accessed 22 August 2024].

Page, E.B., 1966. The imminence of... grading essays by computer. *The Phi Delta Kappan*, 47(5), pp.238-243.

Powers, D.E., Burstein, J., Chodorow, M., Fowles, M.E. and Kukich, K., 2002. Stumping e-rater: Challenging the validity of automated essay scoring. *Computers in Human Behavior*, 18(2), pp.103-134.

Ribeiro, M.T., Singh, S. and Guestrin, C., 2016. "Why should I trust you?": Explaining the predictions of any classifier. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp.1135-1144.

Rodriguez, P., Wang, Y. and Litman, D., 2021. Argument Mining: A Survey. *Computational Linguistics*, 45(4), pp.765-818.

Stevenson, M. and Phakiti, A., 2014. The Effects of Computer-assisted Language Learning (CALL) on English Language Learners in Japanese Universities: A Meta-analysis. *Language Learning & Technology*, 18(3), pp.78-107.

Suen, H.K., 2014. Peer assessment for massive open online courses (MOOCs). *International Review of Research in Open and Distributed Learning*, 15(3), pp.312-327.

Wilson, K.M., Scalise, K. and Gochyyev, P., 2021. The Impact of Automated Essay Scoring on Student Writing Processes and Outcomes. *Journal of Educational Computing Research*, 59(5), pp.933-963.

Zhang, X. and Litman, D., 2020. Coarse-to-Fine Pre-Training for Predicting Argumentative Writing Quality. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05), pp.9592-9599.

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of NAACL-HLT 2019.

He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with disentangled attention. In International Conference on Learning Representations.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., ... & Stoyanov, V. (2021). RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations.

Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R., & Le, Q. V. (2022). XLNet: Generalized autoregressive pretraining for language understanding. In Advances in Neural Information Processing Systems.

APPENDIX A

```
!pip install -q matplotlib
```

```
[ ] !pip  install -q seaborn
```

```
[ ] !pip install -q nltk
```

Importing modules and dependencies

```
[ ] import numpy as np
    import pandas as pd
    import nltk
    import matplotlib.pyplot as plt
    import seaborn as sns
    import re
    import pickle
    from sklearn.feature_extraction.text import CountVectorizer
    from sklearn.model_selection import train_test_split
    from sklearn.ensemble import RandomForestRegressor
    from sklearn.svm import SVR
    from sklearn.metrics import mean_squared_error
    from sklearn.linear_model import LinearRegression
    from nltk.corpus import stopwords
    from nltk.tokenize import word_tokenize
    import warnings
    warnings.filterwarnings("ignore", category=FutureWarning)
    nltk.download('stopwords')
    nltk.download('punkt')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\uniab\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\uniab\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt.zip.
True
```

Load the dataset

```
df = pd.read_csv("Dataset/training_set_rel3.tsv", sep='\t', encoding='ISO-8859-1');
df.head()
```

	essay_id	essay_set	essay	rater1_domain1	rater2_domain1	rater3_domain1	domain1_score
0	1	1	Dear local newspaper, I think effects computer...	4	4	NaN	8
1	2	1	Dear @CAPS1 @CAPS2, I believe that using compu...	5	4	NaN	9
2	3	1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more peopl...	4	3	NaN	7
			Dear Local Newspaper				

```
df.describe()
```

	essay_id	essay_set	rater1_domain1	rater2_domain1	rater3_domain1	domain1_score
count	12976.000000	12976.000000	12976.000000	12976.000000	128.000000	12976.000000
mean	10295.395808	4.179485	4.127158	4.137408	37.828125	6.800247
std	6309.074105	2.136913	4.212544	4.264330	5.240829	8.970705
min	1.000000	1.000000	0.000000	0.000000	20.000000	0.000000
25%	4438.750000	2.000000	2.000000	2.000000	36.000000	2.000000
50%	10044.500000	4.000000	3.000000	3.000000	40.000000	3.000000
75%	15681.250000	6.000000	4.000000	4.000000	40.000000	8.000000
max	21633.000000	8.000000	30.000000	30.000000	50.000000	60.000000

8 rows × 27 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12976 entries, 0 to 12975
Data columns (total 28 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   essay_id        12976 non-null  int64
 1   essay_set       12976 non-null  int64
 2   essay           12976 non-null  object
 3   rater1_domain1  12976 non-null  int64
 4   rater2_domain1  12976 non-null  int64
 5   rater3_domain1  128 non-null    float64
 6   domain1_score   12976 non-null  int64
 7   rater1_domain2  1800 non-null   float64
 8   rater2_domain2  1800 non-null   float64
 9   domain2_score   1800 non-null   float64
 10  rater1_trait1   2292 non-null   float64
 11  rater1_trait2   2292 non-null   float64
 12  rater1_trait3   2292 non-null   float64
 13  rater1_trait4   2292 non-null   float64
 14  rater1_trait5   723 non-null    float64
 15  rater1_trait6   723 non-null    float64
 16  rater2_trait1   2292 non-null   float64
 17  rater2_trait2   2292 non-null   float64
 18  rater2_trait3   2292 non-null   float64
 19  rater2_trait4   2292 non-null   float64
 20  rater2_trait5   723 non-null    float64
 21  rater2_trait6   723 non-null    float64
 22  rater3_trait1   128 non-null    float64
 23  rater3_trait2   128 non-null    float64
 24  rater3_trait3   128 non-null    float64
 25  rater3_trait4   128 non-null    float64
 26  rater3_trait5   128 non-null    float64
 27  rater3_trait6   128 non-null    float64
dtypes: float64(22), int64(5), object(1)
memory usage: 2.8+ MB
```

```
df.shape
```

```
(12976, 28)
```

```
df.isnull().sum()
```

```
essay_id              0
essay_set             0
essay                 0
rater1_domain1        0
rater2_domain1        0
rater3_domain1    12848
domain1_score         0
rater1_domain2    11176
rater2_domain2    11176
domain2_score     11176
rater1_trait1     10684
rater1_trait2     10684
rater1_trait3     10684
rater1_trait4     10684
rater1_trait5     12253
rater1_trait6     12253
rater2_trait1     10684
rater2_trait2     10684
rater2_trait3     10684
rater2_trait4     10684
rater2_trait5     12253
rater2_trait6     12253
rater3_trait1     12848
rater3_trait2     12848
rater3_trait3     12848
rater3_trait4     12848
rater3_trait5     12848
rater3_trait6     12848
dtype: int64
```

```
df[df['essay_set']==5]
```

	essay_id	essay_set	essay	rater1_domain1	rater2_domain1	rater3_domain1	domain1_score
7079	11827	5	In this memoir of Narciso Rodriguez, @PERSON3'...	2	2	NaN	2
7080	11828	5	Throughout the excerpt from Home the Blueprint...	2	2	NaN	2
7081	11829	5	The mood the author created in the memoir is I...	3	3	NaN	3
7082	11830	5	The mood created by the author is showing how ...	1	0	NaN	1
7083	11831	5	The mood created in the memoir is happiness an...	2	3	NaN	3
...	...	...	...	...	...	...	...
8879	13627	5	The mood of this memoir is nonfiction. The moo...	2	1	NaN	2
			The mood was				

```
df.dropna(axis=1,inplace=True)
```

```
df.shape
```

```
(12976, 6)
```

```
df.head()
```

	essay_id	essay_set	essay	rater1_domain1	rater2_domain1	domain1_score
0	1	1	Dear local newspaper, I think effects computer...	4	4	8
1	2	1	Dear @CAPS1 @CAPS2, I believe that using compu...	5	4	9
2	3	1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more peopl...	4	3	7
3	4	1	Dear Local Newspaper, @CAPS1 I have found that...	5	5	10
4	5	1	Dear @LOCATION1, I know having computers has a...	4	4	8

```
df['essay_set'].value_counts()
```

```
essay_set
5    1805
2    1800
6    1800
1    1783
4    1770
3    1726
7    1569
8     723
Name: count, dtype: int64
```

```
df.shape
```

```
(12976, 4)
```

```
df.head(15)
```

	essay_id	essay_set	essay	domain1_score
0	1	1	Dear local newspaper, I think effects computer...	8
1	2	1	Dear @CAPS1 @CAPS2, I believe that using compu...	9
2	3	1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more peopl...	7
3	4	1	Dear Local Newspaper, @CAPS1 I have found that...	10
4	5	1	Dear @LOCATION1, I know having computers has a...	8
5	6	1	Dear @LOCATION1, I think that computers have a...	8
6	7	1	Did you know that more and more people these d...	10
7	8	1	@PERCENT1 of people agree that computers make ...	10
8	9	1	Dear reader, @ORGANIZATION1 has had a dramatic...	9
9	10	1	In the @LOCATION1 we have the technology of a ...	9
10	11	1	Dear @LOCATION1, @CAPS1 people acknowledge the...	8
11	12	1	Dear @CAPS1 @CAPS2 I feel that computers do ta...	8
12	13	1	Dear local newspaper I raed ur argument on the...	7
13	14	1	My three detaileds for this news paper article...	6
14	15	1	Dear, In this world today we should have every...	6

```
min(df['domain1_score'])
```

```
0
```

```
max(df['domain1_score'])
```

```
60
```

```
df.kurtosis()
```

```
essay_id        -1.174375
essay_set       -1.151431
domain1_score    6.077899
dtype: float64
```

```
#Check Skewness
sns.distplot(df['domain1_score'])
```

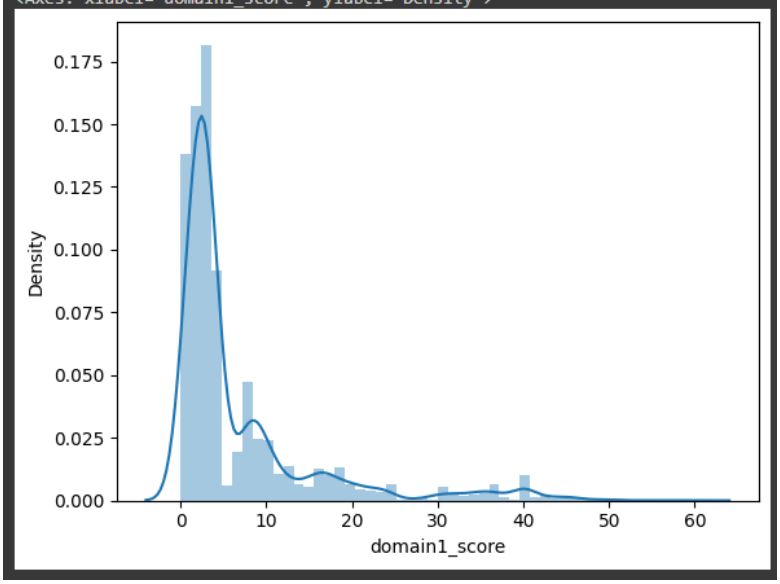
```
C:\Users\uniab\AppData\Local\Temp\ipykernel_16052\114848998.py:2: UserWarning

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(df['domain1_score'])
<Axes: xlabel='domain1_score', ylabel='Density'>
```



```
df[df['essay_set']==5]['domain1_score']

7079    2
7080    2
7081    3
7082    1
7083    3
       ..
8879    2
8880    0
8881    4
8882    3
8883    2
Name: domain1_score, Length: 1805, dtype: int64
```

```
min_range = [2,1,0,0,0,0,0,0]
max_range = [12,6,3,3,4,4,30,60]

def normalize(x,mi,ma):
    print("Before Normalization: "+str(x))
    x = (x-mi)/(ma-mi)
    print("After Normalization : "+str(x))
    return round(x*10)

df['final_score']=df.apply(lambda x:normalize(x['domain1_score'],min_range[x['essay_set']-1],max_range[x['essay_set']-1]),axis=1)
```

```
Before Normalization: 8
After Normalization : 0.6
Before Normalization: 9
After Normalization : 0.7
Before Normalization: 7
After Normalization : 0.5
Before Normalization: 10
After Normalization : 0.8
Before Normalization: 8
After Normalization : 0.6
Before Normalization: 8
After Normalization : 0.6
Before Normalization: 10
After Normalization : 0.8
Before Normalization: 10
After Normalization : 0.8
Before Normalization: 9
After Normalization : 0.7
Before Normalization: 9
After Normalization : 0.7
Before Normalization: 8
After Normalization : 0.6
Before Normalization: 8
After Normalization : 0.6
Before Normalization: 7
After Normalization : 0.5
Before Normalization: 6
After Normalization : 0.4
Before Normalization: 6
After Normalization : 0.4
Before Normalization: 12
After Normalization : 1.0
```

```
df.describe()
```

	essay_id	essay_set	domain1_score	final_score
count	12976.000000	12976.000000	12976.000000	12976.000000
mean	10295.395808	4.179485	6.800247	5.849877
std	6309.074105	2.136913	8.970705	2.466040
min	1.000000	1.000000	0.000000	0.000000
25%	4438.750000	2.000000	2.000000	4.000000
50%	10044.500000	4.000000	3.000000	6.000000
75%	15681.250000	6.000000	8.000000	8.000000
max	21633.000000	8.000000	60.000000	10.000000

```
df.drop('domain1_score',axis=1,inplace=True)
```

```
df.head()
```

	essay_id	essay_set	essay	final_score
0	1	1	Dear local newspaper, I think effects computer...	6
1	2	1	Dear @CAPS1 @CAPS2, I believe that using compu...	7
2	3	1	Dear, @CAPS1 @CAPS2 @CAPS3 More and more peopl...	5
3	4	1	Dear Local Newspaper, @CAPS1 I have found that...	8
4	5	1	Dear @LOCATION1, I know having computers has a...	6

Text PRE_PROCESSING using various techniques

```
[ ] def clean_essay(essay):
        x=[]
        for i in essay.split():
            if i.startswith("@"):
                continue
            else:
                x.append(i)
        return ' '.join(x)

    df['essay'] = df['essay'].apply(lambda x:clean_essay(x))
```

```
[ ] stop_words = set(stopwords.words('english'))
    def remove_stop_words(essay):
        word_tokens = word_tokenize(essay)
        filtered_sentence = []
        for w in word_tokens:
            if w not in stop_words:
                filtered_sentence.append(w)
        return ' '.join(filtered_sentence)

    df['clean_essay'] = df['essay'].apply(lambda x:remove_stop_words(x))
```

```
[ ] def remove_puncs(essay):
        essay = re.sub("[^A-Za-z ]","",essay)
        return essay

    df['clean_essay'] = df['clean_essay'].apply(lambda x:remove_puncs(x))
```

```
df.head()
```

	essay_id	essay_set	essay	final_score	clean_essay
0	1	1	Dear local newspaper, I think effects computer...	6	Dear local newspaper I think effects computer...
1	2	1	Dear I believe that using computers will benef...	7	Dear I believe using computers benefit us many...
2	3	1	Dear, More and more people use computers, but ...	5	Dear More people use computers everyone agre...
3	4	1	Dear Local Newspaper, I have found that many e...	8	Dear Local Newspaper I found many experts say...
4	5	1	Dear I know having computers has a positive ef...	6	Dear I know computers positive effect people ...

```
import re
import nltk

# Download the required NLTK resources
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt')
```

```
df.head()
```

	essay_id	essay_set	essay	final_score	clean_essay
0	1	1	Dear local newspaper, I think effects computer...	6	Dear local newspaper I think effects computer...
1	2	1	Dear I believe that using computers will benef...	7	Dear I believe using computers benefit us many...
2	3	1	Dear, More and more people use computers, but ...	5	Dear More people use computers everyone agre...
3	4	1	Dear Local Newspaper, I have found that many e...	8	Dear Local Newspaper I found many experts say...
4	5	1	Dear I know having computers has a positive ef...	6	Dear I know computers positive effect people ...

processed Dataset with clean corpus of prompts and tokens

```
pro_data = df.copy()
pro_data['char_count'] = pro_data['essay'].apply(noOfChar)
pro_data['word_count'] = pro_data['essay'].apply(noOfWords)
pro_data['sent_count'] = pro_data['essay'].apply(noOfSent)
pro_data['avg_word_len'] = pro_data['essay'].apply(avg_word_len)
pro_data['spell_err_count'] = pro_data['essay'].apply(check_spell_error)
pro_data['noun_count'], pro_data['adj_count'], pro_data['verb_count'], pro_data['adv_count'] = zip(*pro_data['essay'].map(count_pos))
pro_data.to_csv("Processed_dataset.csv")
```

```
prep_df = pd.read_csv("Processed_dataset.csv")
prep_df.drop('Unnamed: 0',inplace=True,axis=1)
prep_df.head()
```

	essay_id	essay_set	essay	final_score	clean_essay	char_count	word_count	sent_count	avg_word_len	spell_err_count	noun_count	adj_count	verb_count	adv_count
0	1	1	Dear local newspaper, I think effects computer...	6	Dear local newspaper I think effects computer...	1441	344	16	4.188953	11	76	75	18	24
1	2	1	Dear I believe that using computers will benef...	7	Dear I believe using computers benefit us many...	1765	413	17	4.273608	21	98	84	20	19
2	3	1	Dear, More and more people use computers, but ...	5	Dear More people use computers everyone agre...	1185	276	14	4.293478	5	76	51	20	16
3	4	1	Dear Local Newspaper, I have found that many e...	8	Dear Local Newspaper I found many experts say...	2284	490	26	4.661224	31	142	96	39	29
4	5	1	Dear I know having computers has a positive ef...	6	Dear I know computers positive effect people ...	2023	469	30	4.313433	18	110	90	32	36

```
df = pd.read_csv('/content/Processed_dataset.csv')
```

Performing Generate Embedding Model(all-MiniLM-L6-v2)

```
embedding_model_name = 'all-MiniLM-L6-v2'
embedding_model = SentenceTransformer(embedding_model_name)
```

```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:89: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens),
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 25.0kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 8.32kB/s]
README.md: 100% 10.7k/10.7k [00:00<00:00, 808kB/s]
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 2.76kB/s]
config.json: 100% 612/612 [00:00<00:00, 41.5kB/s]
model.safetensors: 100% 90.9M/90.9M [00:00<00:00, 166MB/s]
tokenizer_config.json: 100% 350/350 [00:00<00:00, 18.9kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 3.76MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 20.5MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 4.55kB/s]
1_Pooling/config.json: 100% 190/190 [00:00<00:00, 6.92kB/s]
```

```
def generate_embedding(text, embedding_model):
    embedding = embedding_model.encode([text], convert_to_numpy=True)
    return embedding[0]

def process_dataset_with_embeddings(df, embedding_model):
    df['embeddings'] = df['clean_essay'].apply(lambda x: generate_embedding(x, embedding_model))
    df['embeddings'] = df['embeddings'].apply(lambda x: ','.join(map(str, x)))
    return df
```

```
df = process_dataset_with_embeddings(df, embedding_model)
```

```
## Generated Embeddings
df['embeddings']
```

	embeddings
0	-0.02942085,-0.008976666,0.028833369,-0.041873...
1	-0.0275625,-0.014471483,0.0057740235,-0.017673...
2	0.0014746368,-0.011871,0.023100575,-0.03276539...
3	-0.00613624,-0.036276743,-0.007253244,-0.03754...
4	-0.041933954,-0.0027036963,-0.019984702,-0.068...
...	...
12971	0.013403864,-0.11702805,0.014795744,0.07596907...
12972	-0.047734536,-0.1139084,0.05125254,0.07518625,...
12973	0.018796187,-0.11628387,0.015407415,0.04768122...
12974	-0.022904221,-0.089778736,0.00537749,-0.039614...
12975	-0.026224118,-0.09451019,-0.0019473579,0.03089...

12976 rows × 1 columns

```
model_name = 'microsoft/deberta-v3-base'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=1)
```

```
tokenizer_config.json: 100%  52.0/52.0 [00:00<00:00, 3.07kB/s]
config.json: 100%  579/579 [00:00<00:00, 49.0kB/s]
spm.model: 100%  2.46M/2.46M [00:00<00:00, 69.3MB/s]
/usr/local/lib/python3.10/dist-packages/transformers/convert_slow_tokenizer.py:562: UserWarning:
  warnings.warn(
pytorch_model.bin: 100%  371M/371M [00:02<00:00, 107MB/s]
Some weights of DebertaV2ForSequenceClassification were not initialized from the model checkpoint
You should probably TRAIN this model on a down-stream task to be able to use it for predictions a
```

Function to generate prompt

```
def generate_prompt(example):
    return f"Score the following essay on a scale of 1 to 10:\n\n{example['clean_essay']}\n\nScore:"
```

```
df['prompt'] = df.apply(generate_prompt, axis=1)
```

```
## Generated Prompt
df['prompt']
```

	prompt
0	Score the following essay on a scale of 1 to 1...
1	Score the following essay on a scale of 1 to 1...
2	Score the following essay on a scale of 1 to 1...
3	Score the following essay on a scale of 1 to 1...
4	Score the following essay on a scale of 1 to 1...
...	...
12971	Score the following essay on a scale of 1 to 1...
12972	Score the following essay on a scale of 1 to 1...
12973	Score the following essay on a scale of 1 to 1...
12974	Score the following essay on a scale of 1 to 1...
12975	Score the following essay on a scale of 1 to 1...

12976 rows × 1 columns

dtype: object

Converted DataFrame to Dataset

```
dataset = Dataset.from_pandas(df)
```

Tokenizing the dataset for fine-tuning

```
def tokenize_function(examples):
    return tokenizer(examples['prompt'], max_length=512, truncation=True, padding='max_length')

tokenized_dataset = dataset.map(tokenize_function, batched=True)

def add_labels(example):
    example['labels'] = float(example['final_score'])
    return example

tokenized_dataset = tokenized_dataset.map(add_labels, batched=False)
```

```
Map: 100%  12976/12976 [00:09<00:00, 1344.79 examples/s]
Map: 100%  12976/12976 [00:02<00:00, 5313.98 examples/s]
```

```
print(tokenized_dataset)
```

```
Dataset({
    features: ['Unnamed: 0', 'essay_id', 'essay_set', 'essay', 'final_score', 'clean_essay', 'char_count', 'word_count', 'sent_count', 'avg_word_len', '
    num_rows: 12976
})
```

Split dataset

```
train_test_split = tokenized_dataset.train_test_split(test_size=0.2)
train_dataset = train_test_split['train']
eval_dataset = train_test_split['test']
```

```
# Define training arguments
training_args = TrainingArguments
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=8
    per_device_eval_batch_size=8,
    warmup_steps=50,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=100,
    evaluation_strategy='epoch',
    save_strategy='epoch',
    load_best_model_at_end=True,
)

# Define trainer
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
)
```

```
/usr/local/lib/python3.10/dist-pa
  warnings.warn(
```

Train the model

```
trainer.train()
```

[3894/3894 1:17:39, Epoch 3/3]

Epoch	Training Loss	Validation Loss
1	2.267300	2.534856
2	1.788800	1.763001
3	1.380700	1.803147

```
TrainOutput(global_step=3894, training_loss=1.9493789163441553, metrics={'train_runtime': 4660.734, 'train_samples_per_second': 6.681, 'train_steps_per_second': 0.835, 'total_flos': 8193351636725760.0, 'train_loss':
1.9493789163441553, 'epoch': 3.0})
```

Download model after fine-tuning

```
!zip -r ./results.zip ./results
from google.colab import files
files.download('./results.zip')
```

```
  adding: results/ (stored 0%)
  adding: results/checkpoint-1298/ (stored 0%)
  adding: results/checkpoint-1298/training_args.bin (deflated 51%)
  adding: results/checkpoint-1298/config.json (deflated 53%)
  adding: results/checkpoint-1298/scheduler.pt (deflated 56%)
  adding: results/checkpoint-1298/model.safetensors (deflated 19%)
  adding: results/checkpoint-1298/optimizer.pt (deflated 48%)
  adding: results/checkpoint-1298/trainer_state.json (deflated 67%)
  adding: results/checkpoint-1298/rng_state.pth (deflated 25%)
  adding: results/checkpoint-3894/ (stored 0%)
  adding: results/checkpoint-3894/training_args.bin (deflated 51%)
  adding: results/checkpoint-3894/config.json (deflated 53%)
  adding: results/checkpoint-3894/scheduler.pt (deflated 56%)
  adding: results/checkpoint-3894/model.safetensors (deflated 19%)
  adding: results/checkpoint-3894/optimizer.pt (deflated 48%)
  adding: results/checkpoint-3894/trainer_state.json (deflated 73%)
```

User - input based prompt essay text Grading

```
def main():
    essay_text = input("Please enter your essay text: ")
    predicted_score = predict_essay_score_deberta(essay_text, model, tokenizer)
    print(f'\nPredicted Score (DeBERTa Model): {predicted_score:}')

if __name__ == "__main__":
    main()
```

```
Please enter your essay text: "Dear @CAPS1, I know some problems have came up where some individuals do not agree with the

Predicted Score (DeBERTa Model): 6
```

Saving the Fine-Tuned Model and Tokenizer

```python
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# Assume that `model` is your fine-tuned model and `tokenizer` is the tokenizer used for fine-tuning

# Specify the directory where you want to save the model and tokenizer
save_directory = "./fine_tuned_model"

# Save the tokenizer
tokenizer.save_pretrained(save_directory)

# Save the model
model.save_pretrained(save_directory)

print(f"Model and tokenizer saved to {save_directory}")
```

```
Model and tokenizer saved to ./fine_tuned_model
```

```python
# prompt: download this ./fine_tuned_model folder locally

!zip -r ./fine_tuned_model.zip ./fine_tuned_model
files.download('./fine_tuned_model.zip')
```

```
  adding: fine_tuned_model/ (stored 0%)
  adding: fine_tuned_model/special_tokens_map.json (deflated 50%)
  adding: fine_tuned_model/added_tokens.json (stored 0%)
  adding: fine_tuned_model/tokenizer.json (deflated 77%)
  adding: fine_tuned_model/config.json (deflated 53%)
  adding: fine_tuned_model/model.safetensors (deflated 19%)
  adding: fine_tuned_model/spm.model (deflated 50%)
  adding: fine_tuned_model/tokenizer_config.json (deflated 73%)
```

style.css

style.css > .container

```css
body {
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 0;
    background-color: #f4f4f4;
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
}

.container {
    background-color: #fffefe;
    padding: 20px 40px;
    border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
    max-width: 600px;
    width: 100%;
    text-align: center;
}

h1 {
    color: #333;
    margin-bottom: 20px;
}

textarea {
    width: 100%;
    padding: 15px;
    margin-top: 10px;
    border-radius: 5px;
    border: 1px solid #ccc;
```

requirements.txt

```
blinker==1.8.2
Brotli @ file:///C:/b/abs_3d36mno480/croot/brotli-split_1714483178642/work
certifi @ file:///C:/b/abs_7bbxxprxir/croot/certifi_1720453609121/work/certifi
charset-normalizer @ file:///croot/charset-normalizer_1721748349566/work
click==8.1.7
colorama==0.4.6
docx==0.2.4
filelock @ file:///C:/b/abs_f2gie28u58/croot/filelock_1700591233643/work
Flask==3.0.3
fsspec==2024.6.1
gmpy2 @ file:///C:/ci/gmpy2_1645456279018/work
huggingface-hub==0.24.5
idna @ file:///C:/b/abs_aad84bnnw5/croot/idna_1714398896795/work
importlib_metadata==8.2.0
itsdangerous==2.2.0
Jinja2 @ file:///C:/b/abs_92fccttino/croot/jinja2_1716993447201/work
lxml==5.2.2
MarkupSafe @ file:///C:/b/abs_ecfdqh67b_/croot/markupsafe_1704206030535/work
mkl-fft @ file:///C:/b/abs_19i1y8ykas/croot/mkl_fft_1695058226480/work
mkl-random @ file:///C:/b/abs_edwkj1_o69/croot/mkl_random_1695059866750/work
mkl-service==2.4.0
mpmath @ file:///C:/b/abs_7833jrbiox/croot/mpmath_1690848321154/work
networkx @ file:///C:/b/abs_e6gi1go5op/croot/networkx_1690562046966/work
numpy @ file:///C:/Users/dev-admin/mkl/numpy_and_numpy_base_1682982345978/work
packaging==24.1
pillow @ file:///C:/b/abs_32o8er3uqp/croot/pillow_1721059447598/work
protobuf==5.27.3
PyPDF2==3.0.1
PySocks @ file:///C:/ci/pysocks_1605287845585/work
python-docx==1.1.2
PyYAML @ file:///C:/b/abs_782o3mbw7z/croot/pyyaml_1698096085010/work
regex==2024.7.24
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
    <title>Essay Grading App</title>
</head>
<body>
    <div class="container">
        <h1>LLM Based Automated Essay Grading</h1>
        <form method="POST" enctype="multipart/form-data">
            <textarea name="essay" rows="10" cols="50" placeholder="Enter your essay here...">{{ essay_text|default('', true) }}</textarea><br>
            <input type="file" name="essay_file"><br>
            <input type="submit" value="Get Score">
        </form>
        {% if feedback %}
        <div class="result">
            <h2>{{ feedback }}</h2> <!-- Displaying the feedback here -->
        </div>
        {% endif %}
        {% if score is not none %}
        <div class="result">
            <h2>Your Essay Score: {{ score }}</h2>
        </div>
        {% endif %}
    </div>
</body>
</html>
```

app.py

app.py > extract_text_from_file

```python
from flask import Flask, request, render_template
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch
from werkzeug.utils import secure_filename
import os
import PyPDF2
import docx

app = Flask(__name__, template_folder='C:/Users/uniab/FREELANCING/LLM_AUTOMATIC GRADING/PAVAN-LLM-FINAL_This_Week/coding/')

# Configurations
UPLOAD_FOLDER = 'uploads/'
ALLOWED_EXTENSIONS = {'txt', 'pdf', 'docx'}
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

model_dir = "results"  # Directory containing the model
tokenizer_dir = "fine_tuned_model"  # Directory containing the tokenizer

tokenizer = AutoTokenizer.from_pretrained(tokenizer_dir)
model = AutoModelForSequenceClassification.from_pretrained(model_dir)

# Function to provide feedback based on score
def get_feedback(score):
    if score > 5.0:
        return "Feedback: Excellent work! Keep up the great writing."
    elif score > 3.0:
        return "Feedback: Good job! However, there's room for improvement in clarity and structure."
    elif score > 1.0:
        return "Feedback: Fair attempt, but consider improving your argumentation, more essay words and grammar."
    else:
        return "Feedback: Needs significant improvement. Focus on organizing your thoughts, more essay words and improving sentence structure."
```

```python
tokenizer = AutoTokenizer.from_pretrained(tokenizer_dir)
model = AutoModelForSequenceClassification.from_pretrained(model_dir)

# Function to provide feedback based on score
def get_feedback(score):
    if score > 5.0:
        return "Feedback: Excellent work! Keep up the great writing."
    elif score > 3.0:
        return "Feedback: Good job! However, there's room for improvement in clarity and structure."
    elif score > 1.0:
        return "Feedback: Fair attempt, but consider improving your argumentation, more essay words and grammar."
    else:
        return "Feedback: Needs significant improvement. Focus on organizing your thoughts, more essay words and improving sentence structure."

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

def extract_text_from_file(filepath):
    ext = filepath.rsplit('.', 1)[1].lower()
    text = ""

    if ext == 'pdf':
        try:
            with open(filepath, 'rb') as f:
                reader = PyPDF2.PdfReader(f)
                for page in range(len(reader.pages)):
                    text += reader.pages[page].extract_text()
        except Exception as e:
            print(f"Error reading PDF file: {e}")
```

```python
    elif ext == 'docx':
        try:
            doc = docx.Document(filepath)
            for para in doc.paragraphs:
                text += para.text
        except Exception as e:
            print(f"Error reading DOCX file: {e}")

    elif ext == 'txt':
        try:
            with open(filepath, 'r') as f:
                text = f.read()
        except Exception as e:
            print(f"Error reading TXT file: {e}")

    return text

@app.route('/', methods=['GET', 'POST'])
def index():
    feedback = ""
    score = None
    essay_text = ""

    if request.method == 'POST':
        if 'essay_file' in request.files:
            file = request.files['essay_file']
            if file and allowed_file(file.filename):
                filename = secure_filename(file.filename)
                filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
```

app.py > index

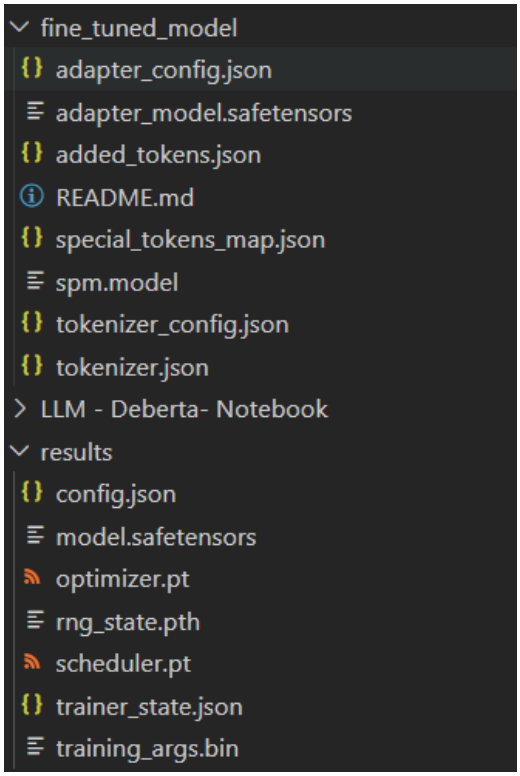
```python
def index():
            file = request.files['essay_file']
            if file and allowed_file(file.filename):
                filename = secure_filename(file.filename)
                filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
                file.save(filepath)
                essay_text = extract_text_from_file(filepath)
                os.remove(filepath)  # Clean up uploaded file after processing
            else:
                feedback = "Invalid file format. Please upload a .pdf, .docx, or .txt file."

        if not essay_text:
            essay_text = request.form.get('essay', '')

        if len(essay_text) < 50:
            feedback = "Essay text is too short. Please write at least 50 characters."
        else:
            inputs = tokenizer(essay_text, return_tensors="pt", truncation=True, padding=True)
            with torch.no_grad():
                outputs = model(**inputs)
                scores = outputs.logits.squeeze().detach().numpy()
                score = round(scores.mean(), 2)  # Averaging across all outputs and rounding off to two decimal places
                feedback = get_feedback(score)

    return render_template('index.html', score=score, essay_text=essay_text, feedback=feedback)

if __name__ == '__main__':
    if not os.path.exists(UPLOAD_FOLDER):
        os.makedirs(UPLOAD_FOLDER)
    app.run(debug=True)
```




```
(base) C:\Users\uniab>conda activate new_env

(new_env) C:\Users\uniab>python app.py
python: can't open file 'app.py': [Errno 2] No such file or directory

(new_env) C:\Users\uniab>cd C:\Users\uniab\FREELANCING\LLM_AUTOMATIC GRADING\PAVAN-LLM-FINAL_This_Week\coding

(new_env) C:\Users\uniab\FREELANCING\LLM_AUTOMATIC GRADING\PAVAN-LLM-FINAL_This_Week\coding>python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 614-782-902
127.0.0.1 - - [13/Aug/2024 14:20:26] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [13/Aug/2024 14:20:26] "GET /static/style.css HTTP/1.1" 404 -
127.0.0.1 - - [13/Aug/2024 14:21:21] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [13/Aug/2024 14:21:21] "GET /static/style.css HTTP/1.1" 404 -
127.0.0.1 - - [13/Aug/2024 14:21:21] "GET /favicon.ico HTTP/1.1" 404 -
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Defau
lt to no truncation.
127.0.0.1 - - [13/Aug/2024 14:22:11] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [13/Aug/2024 14:22:11] "GET /static/style.css HTTP/1.1" 404 -
```

LLM Based Automated Essay Grading

Dear Local @CAPS1, @CAPS2 you believe how many people have computers in our world? Way to many! I think that computers do affect people because with computers, people could @CAPS3 less, get nasty to parents when busy, and get way to addicted to games. First, @CAPS3 is a big role in life and if you don't @CAPS3, it @CAPS2 lead to obesity, laziness, and forget full memory. People that sit near a computer all day are addicts. @CAPS3 is the key for someones body. People could alteast get off for @NUM1 hours and @CAPS3 instead of being so addicted to computers. Second, people that get addicted to thier computers @CAPS2 be nasty if their iterupted. An example of this is when your parents come in your room and tell you to do something then but parents or. I was kids who was addicted to computers I would yell at him back, @CAPS4 them as I like. I have a computer but. and I'm deffinetty not addicted. Anger problems in kids who are addicted are some real big problems. You have them. Last but not games on the computer are just like games. Once you have but it leads to non stop playing. The game get's addicting and kids you it because you always want to beat the game, to back and sometime the game will burn out. Games @CAPS2 lead to being nasty and ignoring. In conclusion, I think that I have explained much of why I think computers are addicting and how they affect people. I hope that you @MONTH1 advise in the paper and hope you feel the same way I do.

Choose File No file chosen

Get Score

Feedback: Excellent work! Keep up the great writing.

Your Essay Score: 5.98

APPENDIX - B

Ethics Checklist

Bournemouth University

Research Ethics Checklist

About Your Checklist	
Ethics ID	57882
Date Created	25/04/2024 14:06:42
Status	Approved
Date Approved	25/04/2024 14:54:08
Risk	Low

Researcher Details	
Name	Pavan Raghavendra Sai Chintala
Faculty	Faculty of Science & Technology
Status	Postgraduate Taught (Masters, MA, MSc, MBA, LLM)
Course	MSc Data Science & Artificial Intelligence
Have you received funding to support this research project?	No

Project Details	
Title	Building an application that uses Large Language Model (LLM) for automatic grading of exams for students
Start Date of Project	15/05/2024
End Date of Project	27/08/2024
Proposed Start Date of Data Collection	25/04/2024
Supervisor	Emili Balaguer-Ballester
Approver	Emili Balaguer-Ballester
Summary - no more than 600 words (including detail on background methodology, sample, outcomes, etc.)	
This project aims to develop an automatic grading system by making use of natural language processing (NLP) technology for streamlining the assessment process for educators and ensure fairness and accuracy in student performance evaluation. As a part of the process, machine learning models and text analysis techniques will be leveraged for analyzing student exam responses, comparing them with predefined reference answers to provide unbiased and consistent grading results.	

Will you have access to confidential corporate or company data (that is not covered by confidentiality terms within an agreement or separate confidentiality agreement)?	No

Storage, Access and Disposal of Research Data	
Where will your research data be stored and who will have access during and after the study has finished.	
Storing research data on my PC and my university's cloud storage offers the best of both worlds. With data on my PC, I have quick access and control, while the university's cloud storage adds security and accessibility. Encrypting my PC and setting access controls on both platforms keeps data safe. It's a balanced approach, meeting both my immediate needs and institutional requirements.	
Once your project completes, will your dataset be added to an appropriate research data repository such as BORDaR, BU's Data Repository?	Yes

Project Proposal Form

Bournemouth University

Department of Computing and Informatics

2023-24 Academic Year Individual Masters Project

Project Proposal Form

Please refer to the **Project Handbook Section 4** when completing this form. Note that your proposal should be your own original work and you must cite sources in line with university guidance on **referencing and plagiarism**[1].

Degree Title: Choose an item.	**Student's Name:** **Pavan Raghavendra Sai Chintala**
	Supervisor's Name: **Emily Balaguer Ballaster**
	Project Title/Area: Building an application that uses Large Language Model (LLM) for automatic grading of exams for students

Section 1: Project Overview

1.1 Problem definition - use <u>one sentence</u> to summarise the problem:

An automatic grading system is required for streamlining and expediting the grading process for educators, which reduces their workload and enables them in allocating more time for teaching responsibilities. Also, a system is required which provides unbiased assessment and consistent for ensuring fairness to all students and at the same time technological advancements are also leveraged.

1.2 Project description - briefly explain your project:

This project aims to develop an automatic grading system by making use of natural language processing (NLP) technology for streamlining the assessment process for educators and ensure fairness and accuracy in student performance evaluation. As a part of the process, machine learning models and text

[1] https://libguides.bournemouth.ac.uk/study-skills-referencing-plagiarism

Bournemouth University

Department of Computing and Informatics

2023-24 Academic Year Individual Masters Project

analysis techniques will be leveraged for analyzing student exam responses, comparing them with predefined reference answers to provide unbiased and consistent grading results.

1.3 Background - please provide brief background information, e.g., client, problem domain, and make reference to the literature (minimum 4-5 sources):

The limitations of traditional grading methods include scalability concerns, consistency in manual grading, lack of feedback that is personalized, bias in manual grading (Fagbohun, et al., 2024). A good grading system should target at improving the process of grading rather than just automation (Gao, et al., 2023), at uncovering limitations of LLMs for this purpose (Schneider, et al., 2023). Automated scoring of self-explainable mathematics is important for task preprocessing, learner response categorization, common misconception identification, tailored feedback creation etc. in digital learning platforms (Nakamoto, et al., 2023.)

1.4 Research Questions

RQ1: Can LLMs be effectively utilized to create automatic grading system of exams on giving predefined reference answers?

RQ2: To what extent can LLMs grade automatically to improve time management of educators?

RQ3: How can LLMs be leveraged to develop intelligent automatic grading systems that provide accurate results and more efficient than manual grading systems?

1.5 Aims and objectives – what are the aims and objectives of your project? should be <u>specific</u> and <u>measurable</u>:

Project Aim:

Develop a prototype system that powers Large Language Models (LLMs) to grade the exam results automatically without manual intervention for making efficient time management to educators by adapting predefined correct answers based on the specific exam patterns.

Specific Objectives:

Objective 1: Design and implement an LLM-based automatic grading module to efficiently and accurately grade student exams, reducing the time and effort required for educators while ensuring reliable and consistent grading results.

Objective 2: Design the system to scale efficiently for handling large volumes of exam papers across different subjects and courses and allow customization of grading criteria and parameters to adapt to diverse subject activities.

Bournemouth University

Department of Computing and Informatics

2023-24 Academic Year Individual Masters Project

Objective 3: Ensure integration with existing learning management systems (LMS) and platforms and implement robust security measures to safeguard student data. Also to ensure compliance with privacy regulations, maintaining confidentiality and integrity throughout the grading process. Also enable the students to track their progress and feedback to enable improvement areas, corrective actions for academic growth and continuous learning.

These objectives are SMART (Specific, Measurable, Achievable, Relevant, and Time-bound):

Specific: Clear action and outcome are outlined for each objective.
Measurable: Quantifiable success metric for each objective.
Achievable: The ambitious objectives are attainable and are in scope.
Relevant: Overall aim can be achieved by the contributions of these objectives.
Time-bound: these objectives are to be achieved within the time limits.

Section 2: Artefact

2.1 What is the artefact that you intend to produce?

The primary artefact of this project will be a prototype system that utilizes Large Language Models (LLMs) to automatically grade the exam results. This system will consist of three main components:

1. NLP (Natural Language Processing) Engine: the underlying algorithms are encompassed by this engine along with the models responsible in understanding and processing natural language text. Pretrained language models are included from Hugging Face will be utilized and are GPT, BERT, RoBERT, and fine-tuned model specific to grading task will be utilized. Student responses will be analyzed by the NLP engine, then compared with the reference answers, and similarity and correctness is determined for each answer.
2. User Interface: UI component is required for providing a platform for interaction of educators and the system. Features such as exam paper uploads, grading setting configurations, view grading results, providing feedback to students, and accessing analytics and reports. The UI should be intuitive, accessible via web browsers or applications, and user-friendly.
3. Data management and integration: storage, retrieval, and exam data management is handled by this component. The exam data include student responses, reference answers, grading criteria, and grading result. This component may integrate with external systems such as databases, learning management systems (LMS), or cloud services (for example AWS S3), for accessing and storing data securely. Data management involves compliance ensuring with privacy regulations, and security measures implementation to protect sensitive information.

2.2 How is your artefact actionable (i.e., routes to implementation and exploitation in the technology domain)?

Bournemouth University

Department of Computing and Informatics

2023-24 Academic Year Individual Masters Project

This LLM-based automatic grading system has a clear path to implementation and exploitation in the domain of technology:

The artefact can be implemented through software development process which includes coding, testing, and deployment of NLP engine, user interface, and data management components. For development, python programming language may be utilized, frameworks like PyTorch or TensorFlow, for building and training NLP models can be utilized. User interface can utilize web technologies such as HTML, CSS, and JS (javascript). Cloud platforms such as Azure, AWS, Google Cloud, with IaaS or PaaS for scalability and reliability.
Integrating with the existing systems of digital learning platforms, student portals, learning management systems, or online assessment tools can be done. This involves APIs development, connectors for enabling data exchange between automatic grading system and other educational system, to allow access to educators for grading functionalities directly within their workflows.
Training and adoption by educators for using automatic grading effectively involves exam papers uploading, grading settings configuration, grading results interpretation, and providing feedback to students. Training sessions, workshops, online tutorials, for training the educators on functionalities and benefits.
Improvement and optimization through feedback mechanisms, data analysis and iteration cycles and helps in enhancing the user experience.

Section 3: Evaluation

3.1 How are you going to evaluate your project artefact?
LLM based automatic grading system can be evaluated in terms of its performance, effectiveness and impact in meeting the objectives and project requirements. The evaluation metrics include accuracy and precision, scalability, user satisfaction, reliability and robustness, alignment with objectives, comparative analysis, feedback incorporation, privacy and security compliance.

By comparing the grading results with human grading and expert evaluations, metrics such as recall, precision, F1score, and accuracy for quantifying the ability of system can be evaluated. Scalability factors such as response time, throughput, and resource utilization for determining the ability of the system can be evaluated.

Feedback can be gathered from educators, students and administrators on their satisfaction. User experience can be assessed by conducting surveys, interviews, and usability testing. Comparing the grading accuracy, efficiency and user satisfaction metrics between the automatic and traditional grading systems. System adherence to data protection laws, encryption practices, access controls, and auditing mechanisms, for safeguarding student confidentiality and privacy can be evaluated.

Bournemouth University

Department of Computing and Informatics

2023-24 Academic Year Individual Masters Project

3.2 How does this project relate to your MSc Programme and your degree title outcomes?

My project on LLM-based automatic grading system directly relates to my MSc in Data Science and AI in several ways, aligning well with the program's expected learning outcomes:

Expertise in Data Analysis and Modeling: The project requires utilizing data science techniques to analyze exam papers to identify grading styles using the LLM. This demonstrates my ability to clean, prepare, and analyze educational data for building predictive models.

Understanding of Machine Learning: The core component of my project is the LLM itself, a powerful machine learning model. Through this project, I gain experience in integrating and utilizing pre-trained LLMs for a specific application in the educational domain.

Problem-Solving Skills: The project tackles a real-world problem in education - the need for automatic grading. I engage data science and AI techniques to propose a solution and develop a prototype system. This showcases my ability to define a problem, develop solutions using appropriate tools, and evaluate its effectiveness.

Communication Skills: My dissertation will involve presenting my research question, methodology, findings, and conclusions clearly. This demonstrates my ability to communicate complex technical concepts to a broader audience, a valuable skill for data scientists working in various fields.

3.3 What are the risks in this project and how are you going to manage them?

some of the potential risks are given below with mitigation strategies.

Model performance and generalization:

Risk: The NLP model may perform well on training data but fail to generalize to unseen exam papers, resulting in inaccurate grading.

Mitigation: Regularly evaluate the model's performance on validation and test datasets to ensure robustness and generalization by using cross validation techniques and transfer learning.

Data Quality and variability:
Risk: Variability in student responses, including spelling errors, ambiguous answers, or diverse writing styles, may lead to incorrect grading.
Mitigation: Implement robust preprocessing techniques to standardize and clean student responses before grading using data augmentation for generating additional training data and improve variability robustness.

Bournemouth University

Department of Computing and Informatics

2023-24 Academic Year Individual Masters Project

Scalability and Performance:
Risk: The system may experience performance bottlenecks or scalability issues when processing a large volume of exam papers or concurrent grading requests
Mitigation: Horizontally scalable system can be designed by using cloud computing for elasticity. Catching mechanisms may be implemented with load balancing and parallel processing for distributing workload efficiently and optimizing the performance.

Integration challenges:
Risk: Integrating the automatic grading system with existing educational systems, such as learning management systems (LMS), may pose technical challenges and compatibility issues
Mitigation: close collaboration with IT teams and administrators for ensuring integration with existing systems, standardized API development, conduct testing and validation of integration points for identifying and resolving compatibility issues in development process.

Section 4: References

4.1 Please provide references if you have used any.

References

Fagbohun, O. et al., 2024. Beyond Traditional Assessment: Exploring the Impact of Large Language Models on Grading Practices.. *J Artif Intell Mach Learn & Data Sci 2024, 2(1),,* pp. 1-8..

Gao, R., Thomas, N. & Srinivasa, A., 2023. *Work in Progress: Large Language Model Based Automatic Grading Study*.. s.l., IEEE., pp. 1-4.

Nakamoto, R. et al., 2023.. Enhancing Automated Scoring of Math Self-Explanation Quality Using LLM-Generated Datasets: A Semi-Supervised Approach.. *Computers, 12(11),,* p. 217..

Schneider, J., Schenk, B., Niklaus, C. & Vlachos, M., 2023. Towards llm-based autograding for short textual answers.. *arXiv preprint arXiv:2309.11508*..

Section 5: Academic Practice and Ethics

Bournemouth University

Department of Computing and Informatics

2023-24 Academic Year Individual Masters Project

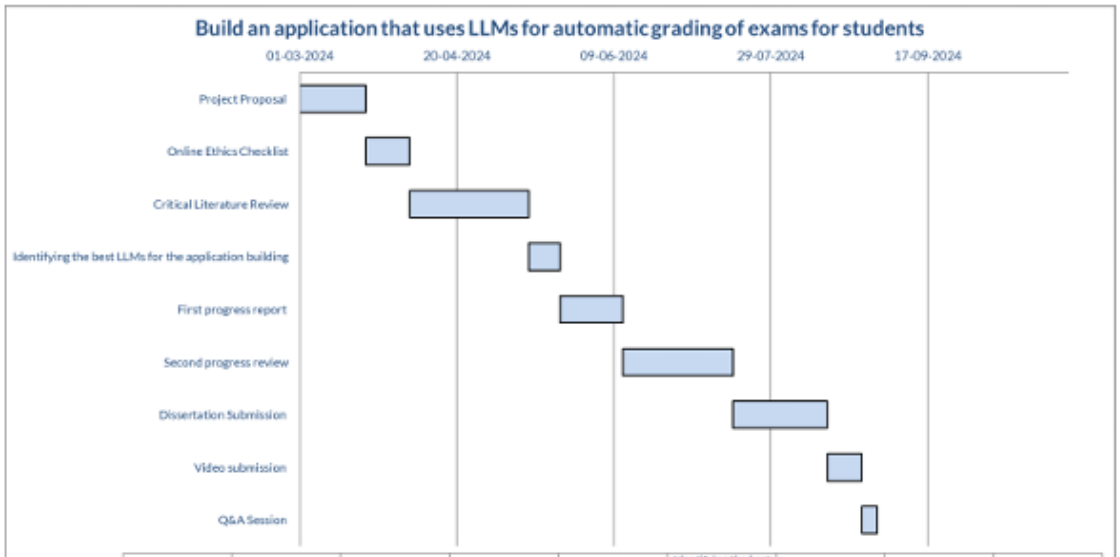
Please delete as appropriate.

5.1 Have you made yourself familiar with, and understand, the University guidance on referencing and plagiarism? Yes / No

5.2 Do you acknowledge that this project proposal is your own work and that it does not contravene any academic offence as specified in the University's regulations? Yes / No

Note: Please complete the research ethics checklist once the proposal has been approved by your supervisor.

Section 6: Proposed Plan (please attach your Gantt chart below)



	Q&A Session	Video submission	Dissertation Submission	Second progress review	First progress report	Identifying the best LLMs for the application building	Critical Literature Review	Online Ethics Checklist	Project Proposal
Starting Date	27-08-2024	16-08-2024	17-07-2024	12-06-2024	23-05-2024	13-05-2024	05-04-2024	22-03-2024	29-02-2024
Days to Complete	5	11	30	35	20	10	38	14	22

Bournemouth University

Department of Computing and Informatics

2023-24 Academic Year Individual Masters Project

NOTE: Do not proceed for ethics form submission if project proposal/feedback looks problematic.